

CS 189/289A — Final Exam Review

Post-Midterm Material (Lectures 17–27, Discussions 8–12, HW 3–5)

UC Berkeley, Spring 2026 — No coding on exam

Closed-book reference. Focus is conceptual + mathematical. Code-only material from homework notebooks is excluded. Discussion / HW cross-references appear inline as “(see Dis X QY)” and “(HWX QY).”

Contents

1	Neural Networks: Foundations	4
1.1	Why neural networks: representation learning on manifolds	4
1.2	The artificial neuron	4
1.3	Feed-forward neural networks	4
1.4	Universal Approximation Theorem	5
1.5	Activation functions and their gradients	6
1.6	Depth vs. width: why “deep” not “fat”?	7
1.7	Worked example: XOR	7
1.8	Quick reference: exam-ready facts	8
2	Backpropagation & Automatic Differentiation	9
2.1	Why backprop: cost comparison vs. alternatives	9
2.2	Computation graph and the bar notation	9
2.3	Backpropagation algorithm	9
2.4	MLP backprop in vector form (HW3 Q3)	10
2.5	Forward-mode vs. reverse-mode autodiff (HW3 Q3d-e)	10
2.6	Worked example: staged computation graph (Dis 9 Q1)	11
2.7	Newton’s method on logistic regression (HW3 Q1)	11
2.8	Gradient descent convergence (Dis 8 Q1)	12
2.9	Coordinate descent (HW3 Q2)	12
2.10	Implicit differentiation (HW3 Q3f-h)	13
2.11	Initialization and batch normalization (lec19)	14
2.12	Pitfalls and quick reference	15
3	Convolutional Neural Networks	16
3.1	Why fully connected layers are wasteful for images	16
3.2	Convolution operation	16
3.3	Stride, padding, dilation, and output dimensions	16
3.4	Worked example: 1D conv as a sparse matrix (Dis 9 Q1)	17
3.5	Worked example: 2D conv (Dis 9 Q2)	17
3.6	Pooling	17

3.7	Backprop through CNNs	18
3.8	Receptive field	18
3.9	Classic architectures	18
3.10	ResNets and residual learning (HW4 Q1–Q7)	18
3.11	Transfer learning	19
3.12	Training-curve concepts (lec20)	19
3.13	Pitfalls and quick reference	20
4	Transformers	21
4.1	Why attention: limits of RNNs and CNNs	21
4.2	Self-attention: Q/K/V	21
4.3	Why divide by $\sqrt{d_k}$? (Dis 10 Q2, HW4 Q14)	22
4.4	Worked example: Q/K/V mechanics (Dis 10 Q1)	22
4.5	Multi-head attention	23
4.6	Positional encoding	23
4.7	Causal masking and decoding (Dis 11 Q2, HW4 Q13)	24
4.8	Encoder, decoder, and cross-attention	24
4.9	Transformer block: residual, layer norm, FFN	25
4.10	KV caching (Dis 11 Q3)	25
4.11	Vision transformers (HW4 Q20–Q21)	26
4.12	Why self-attention beats RNN and CNN (HW4 Q19)	26
4.13	Pitfalls and quick reference	26
5	LLM Training & Applications	28
5.1	Causal language modeling: the training objective	28
5.2	Tokenization	28
5.3	Decoder-only transformer architecture	28
5.4	Pre-training scale and compute	29
5.5	Evaluation metrics (HW4 Q9)	29
5.6	Post-training: fine-tuning	29
5.7	RLHF and instruction tuning	30
5.8	In-context learning and prompting	30
5.9	Retrieval-Augmented Generation (RAG)	31
5.10	Tool use and agents	31
5.11	Vision-language models (Dis 12 Q1)	31
5.12	Pitfalls and quick reference	32
6	Self-Supervised Learning & Generative Models	33
6.1	The SSL premise	33
6.2	Catalogue of pretext tasks (Dis 12 Q2a)	33
6.3	Contrastive learning and InfoNCE (HW5 Q1)	33
6.4	Generative SSL: AR vs. diffusion (Dis 12 Q3)	35
6.5	Diffusion model theory (HW5 Q diffusion)	35
6.6	Flow matching (HW5 flow-matching problem)	36
6.7	Pitfalls and quick reference	37
7	AI for Protein Engineering	38
7.1	Why proteins are hard	38

7.2	Structure prediction: AlphaFold2	38
7.3	Design = conditional generation	38
7.4	Why diffusion/flow are well-suited (continuous)	39
7.5	Discrete-space challenges	39
7.6	Inductive biases for proteins	39
7.7	The pathology of naive ML-based design	39
7.8	Pitfalls and quick reference	40
8	LLMs, Agents, & Environments	41
8.1	Taxonomy: LLM, tool-use, RAG, workflow, agent	41
8.2	Three eras of LLM systems	41
8.3	The long-horizon problem	41
8.4	Environments and Terminal-Bench	42
8.5	Skills: hierarchical capability composition	42
8.6	GEPA: prompt optimization via natural-language reflection	42
8.7	Pitfalls and quick reference	43
9	Information Theory & the Information Bottleneck	44
9.1	Shannon information and entropy	44
9.2	KL divergence and Gibbs' inequality	44
9.3	Maximum entropy and Kraft's bound (HW3 Q4b)	45
9.4	Mutual information	45
9.5	Rate-distortion theory	46
9.6	The Information Bottleneck (HW3 Q4c-d)	46
9.7	The IB identity (HW3 Q4d-i)	46
9.8	The information plane and SGD's two phases	47
9.9	Why noise destroys irrelevant information first (HW3 Q4g)	47
9.10	Pitfalls and quick reference	47
	Appendix A: Worked Problems	49

1 Neural Networks: Foundations

The agenda of this section: motivate why neural networks are needed at all, define the building blocks (neurons, layers), state and prove the universal approximation theorem, catalog the activation functions you must manipulate on the exam, and explain why depth (rather than width) is preferred in practice.

1.1 Why neural networks: representation learning on manifolds

Real-world high-dimensional data — images, audio, text — almost always lies on a low-dimensional manifold inside its huge ambient input space. A 64×64 grayscale image is a vector $\mathbf{x} \in \mathbb{R}^{4096}$, but the actual degrees of freedom of (say) a digit photograph — pose, position, illumination, line thickness — number maybe a dozen.

If we model with a fixed basis (polynomials, RBFs, kernel features) the number of basis elements explodes with the ambient dimension D , not with the manifold dimension. **Representation learning** replaces this fixed basis with a learned, data-dependent one: the hidden activations of a neural network. Each hidden unit acts as a basis function whose location and orientation in input space is fit during training. Complexity then scales with the manifold's intrinsic dimension rather than D .

Pitfall 1.1 (Manifold reasoning is not a theorem). The universal-approximation theorem does not give complexity scaling with the manifold dimension. It says *some* network can match the function; it does not say it can do so with few parameters. The manifold story is empirical and architectural (depth, weight sharing, etc.), not a theorem.

1.2 The artificial neuron

Definition 1.2 (Artificial neuron). An artificial neuron with input $\mathbf{x} \in \mathbb{R}^D$, weight vector $\mathbf{w} \in \mathbb{R}^D$, and bias $w_0 \in \mathbb{R}$ outputs

$$y = f\left(w_0 + \sum_{d=1}^D w_d x_d\right) = f(\mathbf{w}^\top \mathbf{x} + w_0),$$

where $f: \mathbb{R} \rightarrow \mathbb{R}$ is the *activation function*. The argument $a = \mathbf{w}^\top \mathbf{x} + w_0$ is the *pre-activation* (or *logit*) and $z = f(a)$ is the *activation*.

A linear neuron (f identity) cannot represent non-linearly separable functions like XOR. The activation f is what makes the model expressive.

1.3 Feed-forward neural networks

Stacking neurons into layers gives a feed-forward network. Adopt the convention $\mathbf{z}^{(0)} = \mathbf{x} \in \mathbb{R}^D$, with superscript (ℓ) indexing layer $\ell = 1, \dots, L$. Each layer applies an affine map and an elementwise activation:

$$\begin{aligned} \mathbf{a}^{(\ell)} &= \mathbf{W}^{(\ell)} \mathbf{z}^{(\ell-1)} + \mathbf{b}^{(\ell)} && \text{(pre-activation)} \\ \mathbf{z}^{(\ell)} &= h^{(\ell)}(\mathbf{a}^{(\ell)}) && \text{(activation)} \end{aligned}$$

where $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$, and n_ℓ is the width of layer ℓ . The output $\mathbf{z}^{(L)} = \mathbf{y} \in \mathbb{R}^K$.

For a one-hidden-layer net $\mathbb{R}^D \rightarrow \mathbb{R}^K$ with hidden width M , output unit k is

$$y_k(\mathbf{x}; \mathbf{W}) = f\left(\sum_{j=1}^M w_{kj}^{(2)} h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right),$$

or compactly, $\mathbf{y} = f(\mathbf{W}^{(2)} h(\mathbf{W}^{(1)} \mathbf{x}))$ if biases are absorbed by appending a constant 1 to inputs.

Pitfall 1.3 (Deep linear networks collapse). If every activation function is the identity (or any linear map), a network of any depth is equivalent to a single linear map:

$$\mathbf{W}^{(L)} \mathbf{W}^{(L-1)} \dots \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}_{\text{eff}} = \mathbf{W}_{\text{eff}} \mathbf{x} + \mathbf{b}_{\text{eff}}.$$

A linear model cannot solve XOR, so neither can any deep linear network. **Nonlinearity is the indispensable ingredient.**

1.4 Universal Approximation Theorem

Theorem 1.4 (Universal Approximation; Cybenko 1989, Funahashi 1989). Let $f : K \subset \mathbb{R}^D \rightarrow \mathbb{R}^K$ be continuous on a compact set K . For every $\epsilon > 0$, there exists a one-hidden-layer feed-forward network F with a non-polynomial activation function (e.g. sigmoid, tanh, ReLU) such that

$$\sup_{\mathbf{x} \in K} \|f(\mathbf{x}) - F(\mathbf{x})\| < \epsilon.$$

The theorem says *some* one-hidden-layer net can match any continuous target. It does **not** tell you (i) how to find the weights, (ii) how wide the hidden layer must be (the width can grow exponentially with D), or (iii) whether a deeper architecture you use in practice will train to find them.

Constructive proof for ReLU on $[0, 1]$ (Dis 8 Q2). Given $f : [0, 1] \rightarrow \mathbb{R}$ continuous and $\epsilon > 0$, uniform continuity gives a $\delta > 0$ with $|x - y| < \delta \Rightarrow |f(x) - f(y)| < \epsilon$. Partition $[0, 1]$ as $0 = x_0 < x_1 < \dots < x_n = 1$ with each $x_k - x_{k-1} < \delta$. Define the piecewise-linear interpolant

$$p(x) = f(x_{k-1}) + m_k(x - x_{k-1}), \quad m_k = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}, \quad x \in [x_{k-1}, x_k].$$

Each value $p(x)$ on $[x_{k-1}, x_k]$ is a convex combination of $f(x_{k-1})$ and $f(x_k)$, so $|f(x) - p(x)| < \epsilon$ on every interval, hence $\sup_{[0,1]} |f - p| < \epsilon$. Now write p exactly as a ReLU sum:

$$F(x) = C + m_1 \text{ReLU}(x) + \sum_{k=1}^{n-1} (m_{k+1} - m_k) \text{ReLU}(x - x_k),$$

choosing C so that $F(x_0) = f(x_0)$. By construction $F'(x) = m_k = p'(x)$ on each open interval (x_{k-1}, x_k) , so $F \equiv p$ on $[0, 1]$, giving $\sup |f - F| < \epsilon$.

How to read this construction. Each new ReLU $\text{ReLU}(x - x_k)$ is a "hinge" that turns on at x_k and contributes slope 1 from there. The coefficient $(m_{k+1} - m_k)$ controls the change in slope at that knot. Adding $n-1$ such hinges builds an arbitrary continuous piecewise-linear function on $[0, 1]$. Generalizing to $\mathbb{R}^D \rightarrow \mathbb{R}^K$ requires D -dimensional ridge functions of the form $\text{ReLU}(\mathbf{w}^\top \mathbf{x} + b)$ as basis elements; the total hidden-unit count is exponential in D in the worst case.

1.5 Activation functions and their gradients

Six activations to know cold. Let a be the pre-activation and $z = f(a)$.

Sigmoid.

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad \sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Range $(0, 1)$. The derivative peaks at 0.25 (at $a = 0$) and decays to 0 for $|a| \rightarrow \infty$. Saturation $\Rightarrow$ vanishing gradients.

tanh.

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} = 2\sigma(2a) - 1, \quad \tanh'(a) = 1 - \tanh^2(a).$$

Range $(-1, 1)$. Zero-centered (better for hidden layers than sigmoid), but still saturates.

ReLU.

$$\text{ReLU}(a) = \max(0, a), \quad \text{ReLU}'(a) = \mathbf{1}\{a > 0\}.$$

Range $[0, \infty)$. Cheap and avoids vanishing gradients on the active half-line.

Leaky ReLU.

$$\text{LeakyReLU}_\alpha(a) = \max(0, a) + \alpha \min(0, a) = \begin{cases} a & a \geq 0 \\ \alpha a & a < 0 \end{cases},$$

small $\alpha \in (0, 1)$ (typically 0.01). Restores a non-zero gradient for $a < 0$ to mitigate dead ReLUs.

Softplus (a.k.a. "soft ReLU").

$$\text{softplus}(a) = \ln(1 + e^a), \quad \text{softplus}'(a) = \sigma(a).$$

Smooth (everywhere differentiable) approximation to ReLU. As $a \rightarrow \infty$, $\text{softplus}(a) \sim a$.

Softmax (output-only). For $\mathbf{a} \in \mathbb{R}^K$,

$$\text{softmax}(\mathbf{a})_k = \frac{e^{a_k}}{\sum_{j=1}^K e^{a_j}}.$$

Used as the *output* activation of multiclass classifiers, not as a hidden activation. It maps to the simplex: $\text{softmax}(\mathbf{a})_k \in (0, 1)$ and $\sum_k \text{softmax}(\mathbf{a})_k = 1$. Jacobian:

$$\frac{\partial \text{softmax}(\mathbf{a})_k}{\partial a_j} = \text{softmax}(\mathbf{a})_k (\delta_{kj} - \text{softmax}(\mathbf{a})_j).$$

Combined with cross-entropy loss $L = -\sum_k t_k \log \text{softmax}(\mathbf{a})_k$ (where $\mathbf{t}$ is a one-hot target), the pre-activation gradient simplifies beautifully:

$$\frac{\partial L}{\partial a_k} = \text{softmax}(\mathbf{a})_k - t_k.$$

This identity is the reason softmax+cross-entropy is the default classification head; the simplification is detailed further in Section 2.

Pitfall 1.5 (Dead ReLU). If a unit's pre-activation is driven negative for every input in your training set, $\text{ReLU}' = 0$ on that unit forever, so its weights never update again. Common after a too-large gradient step or bad initialization. Leaky ReLU / ELU / careful initialization are standard fixes.

Pitfall 1.6 (Saturating activations and vanishing gradients). σ' and $\tanh'$ are bounded by 0.25 and 1 respectively, and decay to 0 at the tails. In a deep stack the chain rule multiplies many such factors together, so the gradient at early layers shrinks geometrically with depth — the *vanishing gradient problem*. This is the core motivation for ReLU-family activations and (later) residual connections (Section 3).

1.6 Depth vs. width: why "deep" not "fat"?

The universal-approximation theorem only requires one hidden layer — *but exponentially wide in D in the worst case*. Depth can make the same function representable with exponentially fewer parameters.

Logic-circuit analogy.

- A 2-layer logic circuit (AND of ORs) computes any Boolean function on n inputs but may need exponentially many gates.
- Some Boolean functions (parity, multiplication) admit polynomial-size *deep* circuits but provably require exponential size at depth 2.
- Substituting "neuron" for "gate" and "ReLU" for AND/OR, an analogous separation is conjectured (and partially proven) for neural networks: depth saves parameters exponentially for some target functions.

So depth is a parameter-efficiency knob, not an expressivity knob. With finite data, parameter savings translate into sample-complexity savings (less to fit $\Rightarrow$ less data needed). This is the empirical observation behind "deep beats wide" on real data.

1.7 Worked example: XOR

This is the canonical example showing nonlinearity matters.

Example 1.7 (XOR is not linearly separable). Consider the four-point dataset

$$(0,0) \rightarrow 0, \quad (0,1) \rightarrow 1, \quad (1,0) \rightarrow 1, \quad (1,1) \rightarrow 0.$$

A linear model $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ trained with squared loss $J(\mathbf{w}, b) = \frac{1}{4} \sum_i (y_i - f(\mathbf{x}_i))^2$ has gradient (computed in lecture)

$$\nabla J = \frac{1}{2} \begin{pmatrix} 2w_1 + w_2 + 2b - 1 \\ w_1 + 2w_2 + 2b - 1 \\ 2w_1 + 2w_2 + 4b - 2 \end{pmatrix},$$

which vanishes at $\mathbf{w}^* = \mathbf{0}$, $b^* = \frac{1}{2}$, predicting the constant $\frac{1}{2}$ on every input. The same is true for any deep *linear* stack.

Example 1.8 (A 1-hidden-layer ReLU net solves XOR). Take a hidden layer of width $M = 2$ with ReLU and a linear output:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}^\top \mathbf{x} + \mathbf{c}), \quad y = \mathbf{w}^\top \mathbf{h} + b,$$

with

$$\mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \quad b = 0.$$

Computing on each input:

$\mathbf{x}$	$\mathbf{W}^\top \mathbf{x} + \mathbf{c}$	$\mathbf{h}$	$y = \mathbf{w}^\top \mathbf{h}$
(0, 0)	(0, -1)	(0, 0)	0
(0, 1)	(1, 0)	(1, 0)	1
(1, 0)	(1, 0)	(1, 0)	1
(1, 1)	(2, 1)	(2, 1)	0

The hidden ReLU "bends" the input space, making XOR linearly separable in $\mathbf{h}$ -space.

A 2-input MLP with M hidden units has $2M + M + M + 1 = 4M + 1$ free parameters: weights $\mathbf{W} \in \mathbb{R}^{2 \times M}$ ($2M$), hidden biases $\mathbf{c} \in \mathbb{R}^M$ (M), output weights $\mathbf{w} \in \mathbb{R}^M$ (M), and output bias b (1). For $M = 2$ that is 9 parameters — more than enough to fit four data points exactly. Only nonlinearity buys the extra expressivity.

Pitfall 1.9 (Network identifiability). The XOR construction above is not unique — many $(\mathbf{W}', \mathbf{c}', \mathbf{w}', b')$ compute the same input/output map. Two networks with different weights can be behaviorally equivalent. The parameters of a network are not identifiable from its input/output behavior alone; this matters for watermarking, model "stealing," and reasoning about what trained weights "mean."

1.8 Quick reference: exam-ready facts

- One-hidden-layer net with non-polynomial activation is a universal approximator on compact sets (Theorem 1.4). Width can be exponential in input dimension; depth often makes it cheaper.
- Deep *linear* stacks collapse to a single linear map. Activations are essential.
- $\sigma'(a) = \sigma(a)(1 - \sigma(a))$, peaks at 0.25. Saturates.
- $\tanh'(a) = 1 - \tanh^2(a)$. Zero-centered, still saturates.
- $\text{ReLU}'(a) = \mathbf{1}\{a > 0\}$. Subject to dead ReLU.
- Softmax Jacobian: $\partial_j \text{softmax}_k = \text{softmax}_k(\delta_{kj} - \text{softmax}_j)$.
- Softmax + cross-entropy: $\partial L / \partial a_k = \text{softmax}_k - t_k$.
- Universal approx on $[0, 1]$ via ReLU: piecewise-linear interpolant of f at a δ -fine grid; $n-1$ ReLU hinges + one linear term + one bias (Dis 8 Q2).

2 Backpropagation & Automatic Differentiation

Backpropagation is the chain rule, applied systematically over a computation graph in dynamic-programming fashion. This section starts from the scalar chain rule, builds up to the general algorithm, derives MLP backprop in vector form, recasts everything in the vector-Jacobian-product (VJP) language of modern autodiff, and then layers on the optimization techniques that appear together with backprop in HW3 (Newton’s method, coordinate descent, implicit differentiation).

2.1 Why backprop: cost comparison vs. alternatives

Suppose the loss $J(\mathbf{w})$ depends on L parameters and N training points. Three candidate ways to compute $\nabla_{\mathbf{w}} J$:

- **Finite differences.** For each of L parameters, evaluate $J(\mathbf{w} \pm \epsilon \mathbf{e}_i)$. Cost per gradient step: $\mathcal{O}(NL^2)$. With $N=10^3$ and $L=10^6$, this is 10^{15} ops per step — infeasible.
- **Symbolic differentiation** (e.g. SymPy). Exact, but “expression swell” makes the symbolic gradient much larger than the forward expression, with many repeated subexpressions.
- **Backpropagation.** Cost per gradient step is $\mathcal{O}(NL)$ — linear in parameters, linear in training points, linear in hidden units. This is the only viable option for modern nets.

2.2 Computation graph and the bar notation

Treat any function as a directed acyclic graph (DAG) of elementary operations. Nodes are intermediate variables v_i ; an edge $v_i \rightarrow v_j$ means v_j uses v_i . The output node is the scalar loss $\mathcal{L}$.

Definition 2.1 (Adjoint / error signal). For each node v in the graph, define the *adjoint*

$$\bar{v} := \frac{\partial \mathcal{L}}{\partial v}.$$

The bar notation isolates “derivative of loss w.r.t. this node” from “the value of this node.”

The terminal adjoint is $\bar{\mathcal{L}} = 1$.

2.3 Backpropagation algorithm

Single-child (chain) graphs. If each node v_i has exactly one child v_{i+1} , the chain rule gives

$$\bar{v}_i = \bar{v}_{i+1} \frac{\partial v_{i+1}}{\partial v_i}.$$

General DAG (multi-child). If v_i has multiple children $\{v_{c_1}, v_{c_2}, \dots\}$, the multivariate chain rule gives the *sum-over-paths* form

$$\boxed{\bar{v}_i = \sum_{c \in \text{children}(i)} \bar{v}_c \frac{\partial v_c}{\partial v_i}} \tag{1}$$

“Each child sends back its own error signal multiplied by the local Jacobian to v_i , and v_i sums the messages.”

The two-phase algorithm.

1. **Forward pass.** Topologically traverse the graph from inputs to loss, computing and *caching* every v_i .
2. **Backward pass.** Set $\bar{\mathcal{L}} = 1$. Traverse the graph in reverse topological order, applying (1).

Why we cache the forward pass. The local partials $\partial v_c / \partial v_i$ in (1) typically depend on the cached forward values v_i (e.g. $\partial \tanh(a) / \partial a = 1 - \tanh^2(a)$ uses the activation, which we computed forward). Without the cache the backward pass would recompute the forward pass.

2.4 MLP backprop in vector form (HW3 Q3)

Single hidden layer:

$$\mathbf{z} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{h} = \sigma(\mathbf{z}), \quad \hat{y} = \mathbf{w}_2^\top \mathbf{h} + b_2, \quad \mathcal{L} = \frac{1}{2}(\hat{y} - y)^2.$$

Working backward from $\mathcal{L}$:

$$\begin{array}{ll} \bar{\hat{y}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} = (\hat{y} - y) & \text{(scalar)} \\ \bar{\mathbf{w}}_2 = \bar{\hat{y}} \mathbf{h} & \text{vector in } \mathbb{R}^m \\ \bar{b}_2 = \bar{\hat{y}} & \text{scalar} \\ \bar{\mathbf{h}} = \bar{\hat{y}} \mathbf{w}_2 & \in \mathbb{R}^m \\ \bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \sigma'(\mathbf{z}) & \text{(elementwise; } \odot = \text{Hadamard)} \\ \bar{\mathbf{W}}_1 = \bar{\mathbf{z}} \mathbf{x}^\top & \in \mathbb{R}^{m \times d} \\ \bar{\mathbf{b}}_1 = \bar{\mathbf{z}} & \in \mathbb{R}^m \end{array}$$

Why $\bar{\mathbf{W}}_1 = \bar{\mathbf{z}} \mathbf{x}^\top$. Each entry $(\mathbf{W}_1)_{ji}$ enters the loss through $z_j = \sum_{i'} (\mathbf{W}_1)_{ji'} x_{i'} + (b_1)_j$ only, with $\partial z_j / \partial (\mathbf{W}_1)_{ji} = x_i$. Then $\partial \mathcal{L} / \partial (\mathbf{W}_1)_{ji} = \bar{z}_j \cdot x_i$, which is exactly the outer product $\bar{\mathbf{z}} \mathbf{x}^\top$.

2.5 Forward-mode vs. reverse-mode autodiff (HW3 Q3d-e)

A more general framing. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ have Jacobian $J_f \in \mathbb{R}^{m \times n}$.

Definition 2.2 (VJP and JVP). • *Vector-Jacobian product (VJP):* for $\mathbf{v} \in \mathbb{R}^m$, compute $\mathbf{v}^\top J_f \in \mathbb{R}^{1 \times n}$. This is **reverse mode / backprop**.

- *Jacobian-vector product (JVP):* for $\mathbf{u} \in \mathbb{R}^n$, compute $J_f \mathbf{u} \in \mathbb{R}^m$. This is **forward mode**.

Both can be computed without ever materializing J_f .

For a composition $\mathcal{L} = f_K \circ f_{K-1} \circ \cdots \circ f_1(\mathbf{x})$ with intermediate dimensions $n \rightarrow d_1 \rightarrow \cdots \rightarrow d_{K-1} \rightarrow 1$, the full Jacobian is a chain product $J = J_{f_K} \cdots J_{f_1} \in \mathbb{R}^{1 \times n}$.

Cost analysis (HW3 Q3d).

- **Forward mode:** start with a basis vector $\mathbf{u} = \mathbf{e}_i$ and compute $J \mathbf{e}_i \in \mathbb{R}$ via K JVPs. To get the full gradient (all n components) you need n such passes — one per input coordinate. Cost $\propto n$.

- **Reverse mode:** start with $v = \bar{\mathcal{L}} = 1$ and compute $v^\top J \in \mathbb{R}^{1 \times n}$ via K VJPs in a *single* pass. Cost independent of n .

Theorem 2.3 (Why reverse mode wins for ML). For scalar-output computations $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (i.e. a loss), reverse-mode autodiff computes the full gradient in $\mathcal{O}(\text{forward cost})$, while forward-mode autodiff costs $n \times$ the forward cost. The factor of n matters: a modern net has $n \sim 10^9$ parameters, but only one scalar loss.

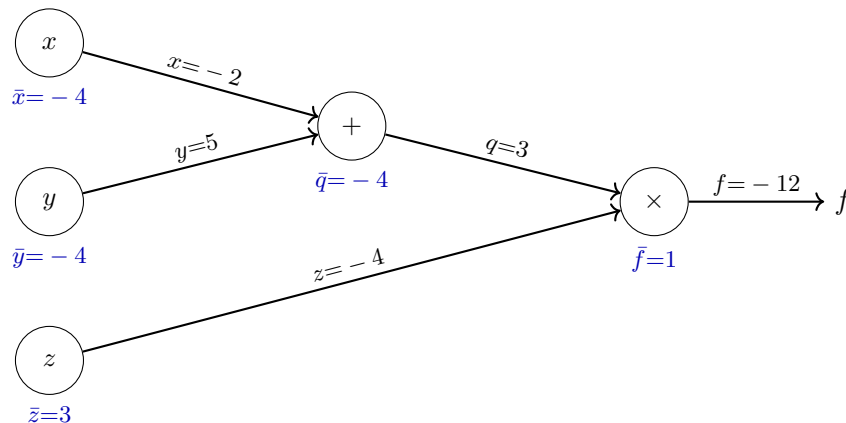
Backprop steps as VJPs (HW3 Q3e). In the one-hidden-layer example above, the backward pass is a sequence of three VJPs:

1. Loss layer: $v = 1$, $f_3 : \hat{y} \mapsto \frac{1}{2}(\hat{y} - y)^2$, Jacobian $\hat{y} - y$. Output: $\bar{\hat{y}} = \hat{y} - y$.
2. Linear layer 2: $v = \bar{\hat{y}}$, $f_2 : \mathbf{h} \mapsto \mathbf{w}_2^\top \mathbf{h} + b_2$, Jacobian $\mathbf{w}_2^\top$. Output: $\bar{\mathbf{h}} = \bar{\hat{y}} \mathbf{w}_2$.
3. Activation + linear layer 1: $v = \bar{\mathbf{h}}$, Jacobian $\text{diag}(\sigma'(\mathbf{z}))$ then a Jacobian w.r.t. $\mathbf{W}_1$, etc.

The pattern: *the upstream gradient is itself the input vector to the next VJP*. This is what makes reverse mode a single sweep.

2.6 Worked example: staged computation graph (Dis 9 Q1)

Let $f(x, y, z) = (x + y)z$. Decompose into $q = x + y$ and $f = qz$. The computation graph:



Forward values in black above each edge; backward gradients in blue below each node.

Forward pass (black, left $\rightarrow$ right): at $(x, y, z) = (-2, 5, -4)$, $q = 3$, $f = -12$.

Backward pass (blue, right $\rightarrow$ left): set $\bar{f} = 1$; then

$$\bar{q} = \bar{f} \cdot z = -4, \quad \bar{z} = \bar{f} \cdot q = 3, \quad \bar{x} = \bar{q} \cdot 1 = -4, \quad \bar{y} = \bar{q} \cdot 1 = -4.$$

The $\bar{q} = -4$ message is forked equally to $\bar{x}$ and $\bar{y}$ because $q = x + y$ has Jacobian $[1, 1]$. This pattern — “copy upstream gradient to each summand” — recurs constantly.

2.7 Newton’s method on logistic regression (HW3 Q1)

Setup: $\mathbf{x}_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$, $s_i = \sigma(\mathbf{x}_i^\top \mathbf{w})$,

$$J(\mathbf{w}) = - \sum_{i=1}^n [y_i \log s_i + (1 - y_i) \log(1 - s_i)] = -\mathbf{y} \cdot \log \mathbf{s} - (\mathbf{1} - \mathbf{y}) \cdot \log(\mathbf{1} - \mathbf{s}).$$

Stack rows: design matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, label vector $\mathbf{y} \in \mathbb{R}^n$, prediction vector $\mathbf{s} \in \mathbb{R}^n$.

Gradient. Use $\sigma'(\gamma) = \sigma(\gamma)(1 - \sigma(\gamma))$. The chain $\partial s_i / \partial \mathbf{w} = s_i(1 - s_i)\mathbf{x}_i$ gives

$$\nabla_{\mathbf{w}} J = \mathbf{X}^\top (\mathbf{s} - \mathbf{y})$$

A clean residual form that drops the $s_i(1 - s_i)$ factor because of the log-loss / sigmoid pairing (the same structural cancellation as softmax + cross-entropy).

Hessian. Differentiating again,

$$\nabla_{\mathbf{w}}^2 J = \mathbf{X}^\top \mathbf{D} \mathbf{X}, \quad \mathbf{D} = \text{diag}(s_i(1 - s_i))_{i=1}^n.$$

Always positive semidefinite (since $\mathbf{D} \succeq 0$), so J is convex — Newton's method has a unique minimizer.

Newton update.

$$\mathbf{w}^{t+1} = \mathbf{w}^t - (\mathbf{X}^\top \mathbf{D}^{(t)} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{s}^{(t)} - \mathbf{y}).$$

Pitfall 2.4 (Newton vs gradient descent on logistic regression). Newton's method converges *quadratically* once close to the optimum, vs. linear convergence for GD. Cost: each step solves a $d \times d$ linear system ($\mathcal{O}(d^3)$ direct, $\mathcal{O}(d^2 k)$ for k CG iterations). For small d this is a free lunch; for $d \gg 10^4$ GD or quasi-Newton (L-BFGS) wins.

2.8 Gradient descent convergence (Dis 8 Q1)

A short reference for the kind of convergence claim you may see on the exam. Assume $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is μ -strongly convex with L -Lipschitz gradient: $\mu I \preceq \nabla^2 f(\mathbf{x}) \preceq LI$ for all $\mathbf{x}$.

Theorem 2.5 (Linear convergence of GD with constant step). For step size $0 < \alpha < 2/L$, the GD iterates $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$ satisfy

$$\|\mathbf{x}_k - \mathbf{x}^*\|_2 \leq M^k \|\mathbf{x}_0 - \mathbf{x}^*\|_2, \quad M = \max\{|1 - \alpha\mu|, |1 - \alpha L|\} < 1.$$

Sketch. Let $T(\mathbf{x}) = \mathbf{x} - \alpha \nabla f(\mathbf{x})$. Its Jacobian is $J_T(\mathbf{x}) = I - \alpha \nabla^2 f(\mathbf{x})$, with eigenvalues in $[1 - \alpha L, 1 - \alpha\mu]$, all of magnitude $\leq M < 1$. Hence T is a contraction; iterating gives the claim. $\square$

Optimal step size. M is minimized when $|1 - \alpha\mu| = |1 - \alpha L|$, giving

$$\alpha^* = \frac{2}{L + \mu}, \quad M^* = \frac{\kappa - 1}{\kappa + 1}, \quad \kappa = L/\mu \text{ (condition number)}.$$

To reach $\|\mathbf{x}_k - \mathbf{x}^*\| \leq \epsilon$ we need $k \in \mathcal{O}(\log(1/\epsilon))$.

2.9 Coordinate descent (HW3 Q2)

A stochastic-gradient style algorithm where each step uses just one coordinate of the gradient. Let

$$g(\mathbf{w}; i) = d \langle \nabla f(\mathbf{w}), \mathbf{e}_i \rangle \mathbf{e}_i, \quad i \sim \text{Unif}\{1, \dots, d\}.$$

This is an unbiased gradient estimator: $\mathbb{E}_i[g(\mathbf{w}; i)] = \nabla f(\mathbf{w})$.

Coordinate-wise smoothness bound. Let $L_i = \sup_{\mathbf{w}} (\nabla^2 f(\mathbf{w}))_{ii}$ and $L_{\max} = \max_i L_i$. Then for any $\mathbf{x}, \mathbf{y}$ with $\text{supp}(\mathbf{x} - \mathbf{y}) \subseteq \{i\}$ (differing only in coordinate i),

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{L_i}{2} \|\mathbf{y} - \mathbf{x}\|_2^2.$$

Convergence with step $\alpha = 1/(L_{\max}d)$. Using the bound above and the random-coordinate identity $\mathbb{E}[\mathbf{e}_i \mathbf{e}_i^\top] = \frac{1}{d} \mathbf{I}$:

$$\mathbb{E}[f(\mathbf{w}_{t+1})] \leq \mathbb{E}\left[f(\mathbf{w}_t) - \frac{1}{2dL_{\max}} \|\nabla f(\mathbf{w}_t)\|_2^2\right].$$

Combined with PL/strong-convexity ($\frac{1}{2} \|\nabla f\|^2 \geq m(f - f^*)$),

$$\mathbb{E}[f(\mathbf{w}_t) - f^*] \leq \left(1 - \frac{m}{dL_{\max}}\right)^t \mathbb{E}[f(\mathbf{w}_0) - f^*].$$

Quadratic example $f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w}$, $\mathbf{A} \succ 0$. $\nabla f(\mathbf{w}) = \mathbf{A} \mathbf{w} - \mathbf{b}$, so coordinate descent on this f updates

$$w_i^{t+1} = w_i^t - \alpha(\mathbf{a}_i^\top \mathbf{w}^t - b_i),$$

where $\mathbf{a}_i$ is the i th row of $\mathbf{A}$. Here $L_i = A_{ii}$, $L_{\max} = \max_i A_{ii}$, $m = \lambda_{\min}(\mathbf{A})$, $L = \lambda_{\max}(\mathbf{A})$.

Cost vs. full GD (HW3 Q2e-f).

- Full GD step: $\mathcal{O}(d^2)$ for matrix-vector $\mathbf{A} \mathbf{w}$. Reach ϵ in $\mathcal{O}((L/m) \log \frac{1}{\epsilon})$ steps. Total $\mathcal{O}(d^2(L/m) \log \frac{1}{\epsilon})$.
- Coord descent step: $\mathcal{O}(d)$ (one row dot product). $\mathcal{O}(dL_{\max}/m) \log \frac{1}{\epsilon}$ steps. Total $\mathcal{O}(d^2(L_{\max}/m) \log \frac{1}{\epsilon})$.
- Always $L_{\max} \leq L$ (since $\lambda_{\max}(\mathbf{A}) = \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \mathbf{A} \mathbf{v} \geq \mathbf{e}_i^\top \mathbf{A} \mathbf{e}_i = A_{ii}$), so coord descent is never worse and can be much better when $\mathbf{A}$'s diagonal dominates.

2.10 Implicit differentiation (HW3 Q3f-h)

Some quantities are defined implicitly — e.g. as the solution to an optimization problem. Let

$$\mathbf{x}^*(\boldsymbol{\theta}) = \arg \min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}, \boldsymbol{\theta}).$$

There is no explicit formula for $\mathbf{x}^*$. But the optimality condition

$$F(\mathbf{x}^*(\boldsymbol{\theta}), \boldsymbol{\theta}) := \nabla_{\mathbf{x}} f(\mathbf{x}^*, \boldsymbol{\theta}) = 0$$

implicitly relates them. Differentiate this w.r.t. $\boldsymbol{\theta}$:

$$\underbrace{\partial_1 F(\mathbf{x}^*, \boldsymbol{\theta})}_{\mathbf{A} \in \mathbb{R}^{d \times d}} \underbrace{\frac{\partial \mathbf{x}^*}{\partial \boldsymbol{\theta}}}_{\mathbf{J} \in \mathbb{R}^{d \times n}} + \underbrace{\partial_2 F(\mathbf{x}^*, \boldsymbol{\theta})}_{\mathbf{B} \in \mathbb{R}^{d \times n}} = 0.$$

For $F = \nabla_{\mathbf{x}} f$, this is

$$\boxed{\frac{\partial \mathbf{x}^*}{\partial \boldsymbol{\theta}} = -[\nabla_{\mathbf{x}}^2 f(\mathbf{x}^*, \boldsymbol{\theta})]^{-1} \partial_{\boldsymbol{\theta}} \nabla_{\mathbf{x}} f(\mathbf{x}^*, \boldsymbol{\theta}).}$$

The implicit function theorem requires $\mathbf{A} = \nabla_{\mathbf{x}}^2 f$ to be invertible.

Ridge example. For $\mathbf{x}^*(\lambda) = \arg \min_{\mathbf{x}} \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 + \frac{\lambda}{2} \|\mathbf{x}\|^2$:

- Optimality: $F(\mathbf{x}, \lambda) = \mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b}) + \lambda\mathbf{x} = 0$.
- $\partial_1 F = \mathbf{A}^\top \mathbf{A} + \lambda I$, $\partial_2 F = \mathbf{x}$.
- $\frac{\partial \mathbf{x}^*}{\partial \lambda} = -(\mathbf{A}^\top \mathbf{A} + \lambda I)^{-1} \mathbf{x}^*$.

This matches differentiating the closed form $\mathbf{x}^*(\lambda) = (\mathbf{A}^\top \mathbf{A} + \lambda I)^{-1} \mathbf{A}^\top \mathbf{b}$ directly using $\frac{d}{d\lambda} \mathbf{M}(\lambda)^{-1} = -\mathbf{M}^{-1} \mathbf{M}' \mathbf{M}^{-1}$.

Why not just unroll? (HW3 Q3g) Alternative: run T steps of GD, then backprop through the unrolled chain.

- Unrolling: *memory* $\mathcal{O}(dT)$ (must cache every iterate), *time* $\mathcal{O}(dT)$ for backward. Memory grows with T .
- Implicit diff: *memory* $\mathcal{O}(d)$ (just the final $\mathbf{x}^*$ and one linear solve), *time* dominated by the linear solve. Memory independent of T .

Pitfall 2.6 (When implicit diff breaks down (HW3 Q3h)). Requires $\mathbf{A} = \nabla_{\mathbf{x}}^2 f$ invertible. Counterexample: $f(x, \theta) = x^4 - \theta x$ has minimizer $x^* = (\theta/4)^{1/3}$. At $\theta = 0$, $f''(0) = 0$ (“flat minimum”), so $\mathbf{A}$ is singular. Geometrically, the response of x^* to small θ is non-Lipschitz ($\propto \theta^{1/3}$). Numerically, near-singular $\mathbf{A}$ gives an error bound $\|\hat{J} - \partial_{\theta} \mathbf{x}^*\| \leq C \|\hat{\mathbf{x}} - \mathbf{x}^*\|$ where C blows up with the condition number of $\mathbf{A}$.

2.11 Initialization and batch normalization (lec19)

Why not initialize to zero. If every weight starts at 0 then by symmetry every hidden unit in a layer computes the same activation and receives the same gradient $\Rightarrow$ they all update identically and remain identical forever. The network is a glorified linear model.

Random small init. Sample $w \sim \mathcal{N}(0, \epsilon_\ell^2)$. *He initialization* for ReLU networks sets $\epsilon_\ell^2 = 2/n_{\text{in}}$ (roughly preserves activation variance through ReLU layers); *Xavier* sets $1/n_{\text{in}}$ (for tanh).

Batch normalization. For pre-activation $a_{km}^{(\ell)}$ at layer ℓ , unit m , training example k in a mini-batch of size K :

$$\hat{a}_{km}^{(\ell)} = \frac{a_{km}^{(\ell)} - \mu_m^{(\ell)}}{\sqrt{(\sigma_m^{(\ell)})^2 + \delta}}, \quad \mu_m^{(\ell)} = \frac{1}{K} \sum_k a_{km}^{(\ell)}, \quad (\sigma_m^{(\ell)})^2 = \frac{1}{K} \sum_k (a_{km}^{(\ell)} - \mu_m^{(\ell)})^2.$$

Then a learnable affine $\gamma \hat{a} + \beta$ restores expressivity. **Training:** μ, σ are mini-batch statistics. **Inference:** use *running* (exponential-moving-average) estimates of μ, σ collected during training. BN’s effect is to keep pre-activations in the gradient “sweet spot” of the nonlinearity, even deep in the network.

Input normalization. For continuous features, always normalize to zero mean and unit variance. *Apply the exact same transform* (using training-set statistics) to validation/test data — never re-fit on test.

2.12 Pitfalls and quick reference

Pitfall 2.7 (Backprop “copy” through a sum). A node $q = \sum_i x_i$ has Jacobian all-ones, so $\bar{x}_i = \bar{q}$ for every i . The upstream gradient is *copied* to each summand.

Pitfall 2.8 (Backprop through an elementwise nonlinearity). $\bar{a} = \bar{z} \odot \sigma'(a)$ (Hadamard, not matmul). A frequent off-by-one: forgetting the elementwise multiplication and matrix-multiplying instead.

Pitfall 2.9 (Subgradient at ReLU kink). $\text{ReLU}'(0)$ is undefined; in practice the autodiff engine sets it to 0 (or sometimes 1). On the exam, write the ReLU derivative as $\mathbf{1}\{a > 0\}$ unless you need a subgradient argument.

Cheat sheet.

- Backprop master equation: $\bar{v}_i = \sum_{c \in \text{ch}(i)} \bar{v}_c \partial v_c / \partial v_i$.
- Reverse mode = single backward pass (cost $\sim$ forward cost). Forward mode = n passes for $f : \mathbb{R}^n \rightarrow \mathbb{R}$.
- Logistic regression gradient: $\mathbf{X}^\top (\mathbf{s} - \mathbf{y})$. Hessian: $\mathbf{X}^\top \text{diag}(s_i(1 - s_i)) \mathbf{X}$.
- GD with $\alpha = 2/(L + \mu)$ has rate $(\kappa - 1)/(\kappa + 1)$, $\kappa = L/\mu$.
- Coord descent has $L_i \leq L = \lambda_{\max}$; one step costs $\mathcal{O}(d)$ for quadratics, vs. $\mathcal{O}(d^2)$ for full GD.
- Implicit diff: $d\mathbf{x}^*/d\boldsymbol{\theta} = -[\nabla_{\mathbf{x}}^2 f]^{-1} \partial_{\boldsymbol{\theta}} \nabla_{\mathbf{x}} f$.
- He init for ReLU: $\mathcal{N}(0, 2/n_{\text{in}})$.

3 Convolutional Neural Networks

CNNs replace fully connected layers' *global template matching* with a *parts-and-compositions* approach: small filters scan the image, looking for local patterns that get reused everywhere. Two ideas drive this: **local connectivity** (each hidden unit sees only a small input window) and **weight sharing** (the same filter is applied at every spatial location). Together they slash parameter counts and bake translation equivariance into the architecture.

3.1 Why fully connected layers are wasteful for images

A fully connected (FC) layer with n_i inputs and n_o outputs has $n_i n_o$ parameters. For a 200×200 RGB image with 1000 hidden units that is 1.2×10^8 weights for a single layer. Worse, every spatial position needs its own weights, so a digit shifted by one pixel appears completely different to the network.

Convolutional layers fix both issues:

- **Local connectivity:** a hidden unit only depends on a small $K \times K$ patch.
- **Weight sharing:** the same $K \times K$ filter is reused at all output positions, so a feature detector learned in one location is immediately useful elsewhere.

3.2 Convolution operation

In ML the “convolution” is actually cross-correlation: no kernel flip.

1D convolution. For input $\mathbf{x} \in \mathbb{R}^D$ and filter $\mathbf{k} \in \mathbb{R}^K$, output

$$y_t = \sum_{\tau=1}^K k_{\tau} x_{t+\tau-1}, \quad t = 1, \dots, D - K + 1.$$

2D convolution. For input image $X \in \mathbb{R}^{H \times W}$ and filter $K \in \mathbb{R}^{k_h \times k_w}$,

$$Y_{i,j} = \sum_{u=1}^{k_h} \sum_{v=1}^{k_w} K_{u,v} X_{i+u-1, j+v-1}.$$

Multi-channel input + multi-filter output. An image with C input channels (e.g. RGB has $C = 3$) and F filters of size $k_h \times k_w$: each filter has shape $k_h \times k_w \times C$, and the layer outputs F feature maps. **Parameter count per layer:** $F \cdot k_h \cdot k_w \cdot C$ (plus F biases). The output spatial size depends on input size, K , stride, and padding; the output *depth* is F .

3.3 Stride, padding, dilation, and output dimensions

Definition 3.1 (Stride and padding). **Stride s :** the filter moves s pixels at each step (skips pixels). **Padding p :** pad the input with p zeros on each side before convolving.

Theorem 3.2 (Output dimension formula). For an input of size W , kernel size K , padding p , and stride s ,

$$W' = \left\lfloor \frac{W - K + 2p}{s} \right\rfloor + 1.$$

Common choices.

- “Valid” (no padding), stride 1: $W' = W - K + 1$. Output shrinks.
- “Same” (zero-pad), stride 1: $p = (K - 1)/2$ for odd K , $W' = W$.
- Stride 2, kernel 3, pad 1: $W' = \lceil W/2 \rceil$. Common downsampling pattern in modern nets.

3.4 Worked example: 1D conv as a sparse matrix (Dis 9 Q1)

Let $\mathbf{x} = [x_1, \dots, x_9]^\top$, kernel $\mathbf{k} = [k_1, k_2, k_3]^\top$.

No padding, stride 1. Output has $9 - 3 + 1 = 7$ entries: $y_1 = k_1x_1 + k_2x_2 + k_3x_3$, $y_2 = k_1x_2 + k_2x_3 + k_3x_4$, $\dots$, $y_7 = k_1x_7 + k_2x_8 + k_3x_9$. As $\mathbf{y} = \mathbf{K}\mathbf{x}$ with $\mathbf{K} \in \mathbb{R}^{7 \times 9}$:

$$\mathbf{K} = \begin{pmatrix} k_1 & k_2 & k_3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & k_1 & k_2 & k_3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & k_1 & k_2 & k_3 & 0 & 0 & 0 & 0 \\ \vdots & & & \ddots & & & & & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & k_1 & k_2 & k_3 \end{pmatrix}.$$

This is a **Toeplitz** (banded) matrix. Convolution is a linear operation; it differs from a generic linear layer only in that the same weights k_1, k_2, k_3 are shared along all rows.

Padding $p = 1$, stride $s = 2$. Output dimension $\lfloor (9 - 3 + 2)/2 \rfloor + 1 = 5$. First entry uses the left zero pad: $y_1 = k_1 \cdot 0 + k_2x_1 + k_3x_2$; second entry: $y_2 = k_1x_2 + k_2x_3 + k_3x_4$.

3.5 Worked example: 2D conv (Dis 9 Q2)

Image $X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$, kernel $K = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, no padding, stride 1.

Kernel size 2×2 , image 3×3 , so output is $(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$. Each entry is the average of a 2×2 window:

$$\begin{aligned} Y_{1,1} &= \frac{1}{4}(1 + 2 + 4 + 5) = 3, & Y_{1,2} &= \frac{1}{4}(2 + 3 + 5 + 6) = 4, \\ Y_{2,1} &= \frac{1}{4}(4 + 5 + 7 + 8) = 6, & Y_{2,2} &= \frac{1}{4}(5 + 6 + 8 + 9) = 7. \end{aligned}$$

So $Y = \begin{pmatrix} 3 & 4 \\ 6 & 7 \end{pmatrix}$. This is a 2×2 **box filter** (a moving average), useful for blurring or as average pooling.

3.6 Pooling

A pooling layer shrinks feature maps by aggregating non-overlapping windows.

- **Max pooling:** $y = \max_{(u,v) \in \text{window}} x_{u,v}$.
- **Average pooling:** $y = \frac{1}{|\text{window}|} \sum x_{u,v}$.

A typical 2×2 pool with stride 2 halves height and width. Pooling also injects **translation invariance**: a small shift of the input often does not change the output of a max-pool over a 2×2 window.

3.7 Backprop through CNNs

Two notes specific to CNNs:

1. **Weight sharing $\Rightarrow$ gradient accumulation.** Each filter weight $K_{u,v}$ is reused at every output position. The gradient $\partial\mathcal{L}/\partial K_{u,v}$ sums contributions across all spatial positions:

$$\frac{\partial\mathcal{L}}{\partial K_{u,v}} = \sum_{i,j} \frac{\partial\mathcal{L}}{\partial Y_{i,j}} X_{i+u-1, j+v-1}.$$

2. **Max-pool $\Rightarrow$ gradient routed to the argmax.** Only the input that achieved the max receives a gradient; the others get 0 (technically a subgradient). On the forward pass, cache the argmax position so the backward pass knows where to route.

3.8 Receptive field

Definition 3.3 (Receptive field). The receptive field of a hidden unit is the set of input pixels that can affect its activation.

For a stack of conv layers with kernel sizes K_ℓ , strides s_ℓ , the receptive field width r_L at layer L satisfies the recursion

$$r_L = r_{L-1} + (K_L - 1) \prod_{\ell < L} s_\ell, \quad r_0 = 1.$$

Implications:

- Without strides or pooling, receptive field grows linearly: $r_L = 1 + L(K - 1)$ for $K \times K$ filters.
- With pooling/strides, it grows much faster — this is why deep networks can “see” the whole image even with 3×3 filters.

3.9 Classic architectures

- **LeNet (1998):** 5×5 filters, average pooling, sigmoid; $\sim 60\text{K}$ parameters; MNIST.
- **AlexNet (2012):** eight layers ($5 \rightarrow 11 \times 11$ early, 3×3 later), max pool, ReLU, dropout, GPU training; ImageNet breakthrough.
- **VGG (2014):** stacked 3×3 filters with max pooling; uniform design; very deep (16/19 layers).
- **ResNet (2015):** residual connections enable training of 50/101/152-layer networks — see next subsection.

A 2015-era empirical observation: parameter count scales with model size, but accuracy gains saturate without architectural innovation. ResNet is the innovation that broke the depth ceiling.

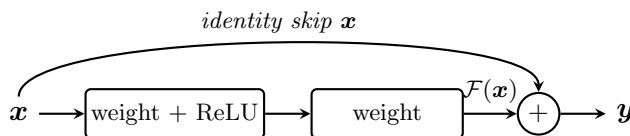
3.10 ResNets and residual learning (HW4 Q1–Q7)

The degradation problem. Empirically, naively-trained 56-layer plain CNNs perform *worse* on training data than 20-layer plain CNNs — not just on test data. This is not overfitting (training error is also worse). It is an *optimization failure*: deep plain networks are hard to train, even when a deeper representation would in principle improve accuracy.

Identity-mapping argument. You can in principle build a deeper network that matches a shallower one’s accuracy: take the shallow net and append identity layers. So the optimal deeper net cannot be worse than the shallow one. The fact that SGD fails to find such a configuration is a limitation of the optimization, not the function class.

Residual block. Replace each block of layers' learned function $\mathcal{H}(\mathbf{x})$ with $\mathcal{F}(\mathbf{x}) + \mathbf{x}$, where $\mathcal{F}$ is the learned residual:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \mathbf{W}) + \mathbf{x}.$$



The lower path computes the residual $\mathcal{F}(\mathbf{x})$; the upper skip arrow carries $\mathbf{x}$ unchanged. They sum at $+$ to give $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$.

The skip path adds $\mathbf{x}$ unchanged. The forward map is $\mathcal{F} + I$; the gradient $\partial \mathbf{y} / \partial \mathbf{x} = \partial \mathcal{F} / \partial \mathbf{x} + I$ contains $+I$ from the skip, ensuring gradient flow. If the optimal block were the identity, the network has only to drive $\mathcal{F} \rightarrow 0$, which is much easier than coercing a stack of nonlinear layers to compute the identity from scratch.

Why residuals help (HW4 Q4).

- **Easier optimization:** the identity is the default; learning $\mathcal{F} = 0$ requires only setting $\mathbf{W} = 0$.
- **Gradient flow:** $\partial \mathbf{y} / \partial \mathbf{x} = \partial \mathcal{F} / \partial \mathbf{x} + I$; the $+I$ term ensures the gradient cannot vanish through the residual branch.

Dimension matching (HW4 Q5–Q6). If $\dim \mathcal{F}(\mathbf{x}) \neq \dim \mathbf{x}$ (e.g. when stride changes spatial size or filter count changes channel depth), three options for the shortcut $\mathbf{x} \rightarrow \mathbf{y}$:

1. **Identity shortcut with zero-padding** for missing channels: no extra parameters.
2. **Projection shortcut only on dim-mismatched blocks:** 1×1 conv just where needed; few extra parameters.
3. **Projection on every block:** 1×1 conv everywhere; most parameters; marginally better accuracy at higher cost.

The paper finds option (2) is the sweet spot. More projections $\Rightarrow$ more memory and FLOPs.

Empirical evidence (HW4 Q7). ResNet-110 trains successfully where a plain-110 network's training error is much higher than its 20-layer counterpart's. The deeper residual networks also generalize better on ImageNet validation, with no degradation as depth grows.

3.11 Transfer learning

A common pattern: take a network pre-trained on ImageNet, freeze the feature-extraction layers, retrain only the classifier head on a new task. Works well because early CNN filters (edges, textures) are largely domain-agnostic. Relevant in HW4's vision setup and revisited for self-supervised learning (Section 6).

3.12 Training-curve concepts (lec20)

Learning curves. Plot training and validation loss vs. epochs. Three regimes: training/validation both decreasing (still learning), training keeps falling but validation rises (overfitting), both plateau (converged or stuck).

Early stopping. Stop training once validation loss stops improving. Implicitly a regularizer: limited training time bounds how far the model can move from the small-norm initialization.

Double descent. Classic bias-variance reasoning predicts a “U-shaped” test error vs. model capacity. Modern over-parameterized nets reveal a second descent: as capacity passes the interpolation threshold (training error hits 0), test error first *rises* (peak at the threshold) then *falls again* for ever-larger models. SGD on a massively over-parameterized net implicitly selects a low-norm solution among the many that fit, which generalizes better than the boundary case.

3.13 Pitfalls and quick reference

Pitfall 3.4 (Output dimension off-by-one). $W' = \lfloor (W - K + 2p)/s \rfloor + 1$, not $\lfloor (W - K + 2p)/s \rfloor$. The +1 catches the first window before any sliding.

Pitfall 3.5 (Confusing convolution and cross-correlation). ML libraries do cross-correlation (no kernel flip). True convolution would flip the kernel. In practice the network learns whichever weights solve the task; the distinction matters only when comparing to signal-processing formulas.

Pitfall 3.6 (Channels are not spatial). A 3×3 conv on an RGB image uses a $3 \times 3 \times 3$ tensor of weights (the third 3 is channels), not a flat 3×3 filter applied to each channel separately. The filter *sums* over input channels.

Pitfall 3.7 (Max-pool gradient is sparse). Only the argmax pixel in each window receives gradient. Average-pool shares gradient evenly. This affects which inputs “feel” a given training step.

Cheat sheet.

- Output size: $W' = \lfloor (W - K + 2p)/s \rfloor + 1$.
- Conv params per layer: $F \cdot k_h \cdot k_w \cdot C$ (filters $\times$ kernel $\times$ in-channels).
- Receptive field grows linearly without stride, multiplicatively with stride.
- Backprop: weight gradients sum over positions; max-pool routes to argmax.
- ResNet block: $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$; identity shortcut preserves gradient.
- ResNet shortcut: identity (option 1) for matching dims, 1×1 projection (option 2) only at dim changes.
- Double descent: test error vs. model capacity dips again past interpolation threshold.

4 Transformers

The transformer is the dominant architecture for sequence data. Its core is the *self-attention* layer, which computes a weighted sum of all positions in a sequence, with weights derived from content. This section builds attention from scratch (Q/K/V), derives the $1/\sqrt{d_k}$ scaling factor, walks through a full numerical example, extends to multi-head attention and positional encodings, and covers encoder/decoder structure, residual+layer-norm composition, KV caching, and vision transformers.

4.1 Why attention: limits of RNNs and CNNs

Sequence models before transformers had two failure modes:

- **RNNs:** sequential computation $\Rightarrow$ no parallelism; long-range dependencies require gradients to flow through many time steps $\Rightarrow$ vanishing gradients.
- **Sequential CNNs (like dilated stacks):** parallelizable, but receptive field grows only linearly (or logarithmically) with depth.

Self-attention is parallelizable across positions and has a path length of $\mathcal{O}(1)$ between any two tokens (every position attends directly to every other position).

4.2 Self-attention: Q/K/V

Given an input matrix $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{model}}}$ where each of the N rows is a token embedding, three learned linear projections produce queries, keys, and values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V,$$

with $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Note: in some formulations (including Dis 10 Q1) the convention is $\mathbf{q}_i = \mathbf{W}_Q^\top \mathbf{x}_i$; the row-stacking versions are equivalent, and the row form $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ is what we use throughout.

Attention scores. The unscaled score between query $\mathbf{q}_i$ and key $\mathbf{k}_j$ is the inner product $z_{ij} = \mathbf{q}_i^\top \mathbf{k}_j = (\mathbf{Q}\mathbf{K}^\top)_{ij}$.

Attention weights. Apply softmax row-wise to get probabilities

$$\mathbf{A} = \text{softmax}_{\text{row}}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{N \times N}.$$

Each row sums to 1; A_{ij} is the weight token i places on token j .

Attention output.

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}_{\text{row}}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \in \mathbb{R}^{N \times d_v}$$

4.3 Why divide by $\sqrt{d_k}$? (Dis 10 Q2, HW4 Q14)

Suppose $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ have iid components $q_i, k_i \sim \mathcal{N}(\mu, \sigma^2)$. Then

$$\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = d_k \mu^2.$$

For the practical case $\mu = 0, \sigma = 1$:

$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = \sum_i \text{Var}(q_i k_i) = \sum_i \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = d_k.$$

So $\mathbf{q}^\top \mathbf{k}$ has standard deviation $\sqrt{d_k}$. Without scaling, the dot product magnitude grows with d_k , pushing softmax inputs into its saturation regime where one entry ≈ 1 and others ≈ 0 . The gradient vanishes there. Dividing by $s = \sqrt{d_k}$ makes

$$\mathbb{E}\left[\frac{\mathbf{q}^\top \mathbf{k}}{s}\right] = 0, \quad \text{Var}\left(\frac{\mathbf{q}^\top \mathbf{k}}{s}\right) = 1,$$

so softmax inputs stay $\mathcal{O}(1)$ at initialization and the gradient flows.

Pitfall 4.1 (Softmax temperature vs. $\sqrt{d_k}$ scaling (Dis 11 Q2c)). The $\sqrt{d_k}$ in attention is a *statistical normalization* chosen so initialization variance is 1. Don't confuse it with a learnable or schedulable temperature T in $\text{softmax}(\mathbf{x}/T)$. Behavior:

- $T \rightarrow 0^+$: distribution collapses to a one-hot at $\arg \max$.
- $T \rightarrow \infty$: distribution becomes uniform.

At inference time, decreasing T (or equivalently scaling logits up) sharpens generation; increasing T smooths it.

4.4 Worked example: Q/K/V mechanics (Dis 10 Q1)

Three input tokens $\mathbf{x}_1 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$, $\mathbf{x}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$, $\mathbf{x}_3 = \begin{pmatrix} 1 \\ -2 \end{pmatrix} \in \mathbb{R}^2$ and weight matrices

$$\mathbf{W}_Q = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -3 \end{pmatrix}, \quad \mathbf{W}_K = \begin{pmatrix} 0 & 0 & -1 \\ -2 & 0 & 1 \end{pmatrix}, \quad \mathbf{W}_V = \begin{pmatrix} 8 & 0 \\ 0 & 9 \end{pmatrix}.$$

Convention here: $\mathbf{q}_i = \mathbf{W}_Q^\top \mathbf{x}_i$, etc.

(a) **Compute Q/K/V.** For example,

$$\mathbf{q}_1 = \mathbf{W}_Q^\top \mathbf{x}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -3 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ -5 \end{pmatrix}.$$

Similarly compute $\mathbf{q}_2, \mathbf{q}_3, \mathbf{k}_i, \mathbf{v}_i$.

(b) **Attention scores from token 3.** $z_{3,j} = \mathbf{q}_3^\top \mathbf{k}_j$ for $j = 1, 2, 3$.

(c) **Weighted value sum.** Given softmaxed weights $a_{3,1} \approx 0.00247$, $a_{3,2} \approx 0.99753$, $a_{3,3} \approx 6.9 \times 10^{-13}$:

$$\mathbf{o}_3 = a_{3,1} \mathbf{v}_1 + a_{3,2} \mathbf{v}_2 + a_{3,3} \mathbf{v}_3 \approx \mathbf{v}_2.$$

Token 3's output is essentially token 2's value vector — token 3 “attends almost entirely to token 2.”

(d) **Matrix form.** Stacking all tokens as rows: $\mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{3 \times 3}$, $\mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{3 \times 3}$, $\mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{3 \times 2}$.

(e) $(\mathbf{QK}^\top)_{ij} = z_{ij}$. $(\mathbf{QK}^\top)_{ij} = \mathbf{q}_i^\top \mathbf{k}_j$ by definition of matrix product.

(f) **Row-wise softmax then V.** Applying softmax row-wise to $\mathbf{QK}^\top$ gives $\mathbf{A}$; then row i of $\mathbf{AV}$ is $\sum_j A_{ij} \mathbf{v}_j$ — exactly the per-token weighted-sum-of-values.

4.5 Multi-head attention

Single-head attention forces a single projection $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ to capture all relational structure between tokens. Multi-head attention runs h parallel attention computations with independent parameters, then concatenates and re-projects:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O, \quad \text{head}_i = \text{Attn}(\mathbf{XW}_Q^{(i)}, \mathbf{XW}_K^{(i)}, \mathbf{XW}_V^{(i)}).$$

Typically each head uses dimension $d_k = d_v = d_{\text{model}}/h$ so the total parameter count and compute roughly match a single-head layer of size d_{model} .

Why multi-head (HW4 Q15).

1. Different heads can specialize in different relations (syntactic vs. semantic, short-range vs. long-range).
2. Provides ensemble-like diversity at no extra cost.

4.6 Positional encoding

Self-attention is permutation-equivariant: shuffling rows of $\mathbf{X}$ shuffles rows of the output identically. Without positional information the model treats sentences as bags of tokens. Three approaches:

Sinusoidal (Vaswani et al.). Add a deterministic position-dependent vector to the input embeddings:

$$\text{PE}_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right).$$

Properties used in the original paper:

- Each position has a unique d -dimensional code.
- Bounded ($\sin, \cos$ range $[-1, 1]$), so does not dwarf token features.
- Relative offsets $pos + \Delta$ can be expressed as a linear function of pos via angle-addition identities.
- Generalizes to sequences longer than seen in training.

Learned positional embeddings. A trainable matrix $\mathbf{P} \in \mathbb{R}^{N_{\text{max}} \times d_{\text{model}}}$ added to inputs. Simple but does not extrapolate beyond N_{max} .

Rotary Position Embeddings (RoPE; Dis 11 Q1). Modern default in LLMs. For a 2D query/key pair $(x_n^{(1)}, x_n^{(2)})$ at position n and parameter θ , define

$$\text{RoPE}(\mathbf{x}_n, n) = \mathbf{R}(n\theta) \mathbf{x}_n, \quad \mathbf{R}(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}.$$

The structural property: dot products between RoPE-encoded vectors depend only on *relative* position. Specifically,

$$(\mathbf{R}(m\theta)\mathbf{x})^\top (\mathbf{R}(n\theta)\mathbf{y}) = \mathbf{x}^\top \mathbf{R}(m\theta)^\top \mathbf{R}(n\theta) \mathbf{y} = \mathbf{x}^\top \mathbf{R}((n-m)\theta) \mathbf{y},$$

using $\mathbf{R}(\alpha)^\top \mathbf{R}(\beta) = \mathbf{R}(\beta - \alpha)$. Hence

$$\text{RoPE}(\mathbf{x}, m)^\top \text{RoPE}(\mathbf{y}, n) = \text{RoPE}(\mathbf{x}, m+k)^\top \text{RoPE}(\mathbf{y}, n+k)$$

for any shift k — positional information is fully relative.

4.7 Causal masking and decoding (Dis 11 Q2, HW4 Q13)

In autoregressive language models, position i may not attend to future positions $j > i$ (otherwise the model would cheat at next-token prediction). Implement by adding a mask before softmax:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}, \quad \mathbf{A} = \text{softmax}_{\text{row}} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M} \right).$$

Softmax sends $-\infty$ entries to 0, so token i assigns no weight to the future.

Reading attention heatmaps (Dis 11 Q2). Four canonical pre-softmax patterns and their post-softmax shapes:

- **All-equal** (constant matrix $\Rightarrow$ uniform attention): every token attends equally to every other token.
- **Strong diagonal** (e.g. 10 on diagonal, 1 off): each token attends mostly to itself.
- **Lower-triangular** (causal mask): token i attends to a uniform mix of $\{1, \dots, i\}$. Used in decoder-only LLMs.
- **Permutation pattern** (single off-diagonal large value per row): each token attends to a specific other token — e.g. “each word looks at the previous word.”

4.8 Encoder, decoder, and cross-attention

The original transformer (Vaswani et al.) is encoder–decoder for machine translation:

Encoder. A stack of identical blocks, each containing: (i) self-attention, (ii) feed-forward MLP, both wrapped in residual+LayerNorm. The encoder reads the entire source sequence and outputs a memory tensor.

Decoder. A stack of blocks, each with three sublayers:

1. **Masked self-attention** over decoder inputs (causal mask).
2. **Cross-attention:** queries from decoder, keys+values from encoder memory.
3. **Feed-forward MLP.**

Where do Q/K/V come from? (HW4 Q16)

- **Encoder self-attention:** Q, K, V all from encoder input.
- **Decoder self-attention:** Q, K, V all from decoder input (causal-masked).
- **Decoder cross-attention:** Q from decoder, K and V from encoder output.

Train vs. inference (HW4 Q17).

- **Training:** *teacher forcing*. Decoder input at position i is the ground-truth token at $i - 1$, regardless of what the model would predict. Causal mask ensures position i cannot see positions $\geq i$ in its input.
- **Inference:** *auto-regressive*. Decoder input at i is the model's own previously emitted token at $i - 1$.

4.9 Transformer block: residual, layer norm, FFN

A single block is

$$\begin{aligned} \mathbf{X}' &= \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X})) && \text{(pre-norm) or} \\ \mathbf{X}' &= \text{LN}(\mathbf{X} + \text{MHA}(\mathbf{X})) && \text{(post-norm; original paper)} \\ \mathbf{X}'' &= \mathbf{X}' + \text{FFN}(\text{LN}(\mathbf{X}')) \end{aligned}$$

where $\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ is a 2-layer MLP applied position-wise (typically $4\times$ wider in the hidden layer than d_{model}).

Layer norm vs. batch norm. LayerNorm normalizes across the *feature* dimension for each token independently, so it works for variable-length sequences and small batches. BatchNorm normalizes across the batch, which makes it sensitive to sequence length and batch size.

Pre-norm vs. post-norm. Modern LLMs use pre-norm because it gives a clean residual “highway”: gradients flow through the $+\mathbf{X}$ path unmodified, easing optimization for very deep stacks.

4.10 KV caching (Dis 11 Q3)

Decoder-only autoregressive generation is the dominant inference pattern. At step n , computing attention for the new token requires $\mathbf{K}_{1:n}, \mathbf{V}_{1:n}$. Naively each step recomputes all keys and values from scratch.

Without caching. For one layer at iteration n :

- Recompute $\mathbf{K}_i, \mathbf{V}_i$ for all $i \leq n$.
- Token $\mathbf{y}_1$ has its keys/values recomputed at every step $n = 1, 2, \dots, N$ — a total of N times.
- Total key projections per layer: $1 + 2 + \dots + N = \mathcal{O}(N^2)$ matrix-vector products.

With caching. Store $\mathbf{K}_{1:n-1}, \mathbf{V}_{1:n-1}$ from the previous step. At step n , compute only $\mathbf{K}_n, \mathbf{V}_n$ and append. Total key projections per layer: $\mathcal{O}(N)$.

Why it matters. For chatbots and code assistants generating $N \sim 10^3$ tokens, KV caching saves $\sim N^2/N = N$ times the work — two to three orders of magnitude. This is the single most important inference-time optimization for transformer LMs.

4.11 Vision transformers (HW4 Q20–Q21)

Tokens from images. Split the image into non-overlapping $P \times P$ patches; flatten each into a vector and project to dimension d_{model} . A $H \times W$ image yields $\lceil H/P \rceil \cdot \lceil W/P \rceil$ tokens. Add positional embeddings, then a standard transformer encoder.

Variable input sizes (HW4 Q21).

- **ResNet:** convolutions are size-agnostic, but the final flatten/global-average-pool fixes the head; you can usually feed any size that's a multiple of the stride.
- **ViT:** number of patches changes with input size, so the positional embedding table must be resized (interpolated). Patch size is the unit of granularity.

4.12 Why self-attention beats RNN and CNN (HW4 Q19)

For sequences of length N and feature dim d :

	Computation/layer	Sequential ops	Max path length
Self-attention	$\mathcal{O}(N^2 d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Recurrent	$\mathcal{O}(N d^2)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Convolutional	$\mathcal{O}(K N d^2)$	$\mathcal{O}(1)$	$\mathcal{O}(\log_K N)$

Attention's quadratic-in- N compute is the price; in return every position talks to every other position in one step. For modest N and modern GPUs this trade is overwhelmingly favorable.

4.13 Pitfalls and quick reference

Pitfall 4.2 (Convention drift on Q/K/V). Different sources use $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ vs. $\mathbf{q}_i = \mathbf{W}_Q^\top \mathbf{x}_i$. The two are transpose-conjugate. Always check the dimensions of $\mathbf{W}_Q$ before computing.

Pitfall 4.3 (Forgetting that PE is added, not concatenated). Standard transformers *add* positional encodings to token embeddings. Concatenation would change the dimensionality of every subsequent layer.

Pitfall 4.4 (Cross-attention K/V come from encoder). In encoder–decoder cross-attention, queries come from the decoder side (the position currently being generated) and keys/values come from the encoder memory (the source). Reversing this is a common bug.

Pitfall 4.5 (Causal mask vs. padding mask). Causal masks prevent attending to the future. Padding masks (used in encoders) prevent attending to padding tokens. They serve different purposes; they may both be active simultaneously in encoder–decoder.

Cheat sheet.

- Attn output: $\text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})\mathbf{V} \in \mathbb{R}^{N \times d_v}$.
- Why $\sqrt{d_k}$: $\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$ for unit-variance components, so $1/\sqrt{d_k}$ normalizes.
- Multi-head: h parallel attentions, each $d_k = d_{\text{model}}/h$, concatenated then re-projected.
- Sinusoidal PE: alternating sin / cos at exponentially varying frequencies; relative shifts are linear in PE.
- RoPE: rotate Q,K by angle proportional to position; dot products depend only on relative position.

- Causal mask: add $-\infty$ to upper triangle pre-softmax.
- KV cache: $\mathcal{O}(N)$ instead of $\mathcal{O}(N^2)$ projections for length- N generation.
- ViT: image $\rightarrow$ patches $\rightarrow$ tokens; standard transformer encoder.

5 LLM Training & Applications

A modern Large Language Model (LLM) is a stack of decoder-only transformer blocks trained at scale to predict the next token. This section covers the training objective, architectural choices specific to LLMs, evaluation metrics, the post-training stages (fine-tuning, RLHF), and the applications patterns: in-context learning, RAG, tool use, and vision-language fusion.

5.1 Causal language modeling: the training objective

Given a token sequence $\mathbf{w} = (w_1, \dots, w_T)$, the LLM models the joint distribution by the chain rule:

$$p_{\theta}(\mathbf{w}) = \prod_{t=1}^T p_{\theta}(w_t \mid w_{<t}).$$

Training minimizes the negative log-likelihood

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(w_t \mid w_{<t}),$$

which is per-token cross-entropy between the predicted distribution $p_{\theta}(\cdot \mid w_{<t})$ over the vocabulary and the one-hot target w_t . This is exactly the objective from Section 1 applied at every position in parallel via the causal mask (Section 4).

5.2 Tokenization

Raw text is mapped to a sequence of integer tokens via a fixed vocabulary (typically $\sim 30\text{K}$ – 100K entries) using subword schemes like Byte-Pair Encoding (BPE), WordPiece, or SentencePiece.

Why subword? Whole-word tokenization has a vocabulary explosion (millions of word forms) and cannot represent novel words. Character tokenization makes sequences too long. Subword tokenization balances: common words become a single token, rare words decompose into subwords. “unhappiness” might become [un, happi, ness].

Effect on context length. A model with N_{ctx} token context corresponds to roughly $0.75N_{\text{ctx}}$ English words (because average ~ 4 chars per English word, ~ 3.5 chars per token).

5.3 Decoder-only transformer architecture

For an LLM, the encoder-decoder split of the original transformer collapses to a single decoder-only stack. Each block:

$$\begin{aligned} \mathbf{X}' &= \mathbf{X} + \text{MaskedMHA}(\text{LN}(\mathbf{X})) \\ \mathbf{X}'' &= \mathbf{X}' + \text{FFN}(\text{LN}(\mathbf{X}')) \end{aligned}$$

Repeated L times. After the final layer, an output projection $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$ produces logits over the vocabulary V , and softmax gives next-token probabilities. The output projection is typically *tied* to the input embedding matrix to save parameters.

LayerNorm vs. RMSNorm. LayerNorm computes $\text{LN}(\mathbf{x}) = \gamma \frac{\mathbf{x} - \mu}{\sigma + \epsilon} + \beta$ with both centering (μ) and scaling (σ). *RMSNorm* drops the centering:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}}.$$

Empirically as effective and slightly faster (no mean computation). Most modern LLMs (LLaMA, Mistral) use RMSNorm.

5.4 Pre-training scale and compute

Scaling laws. Empirically, test loss decreases as a power law in model size N , dataset size D , and compute C :

$$\mathcal{L}(N, D) \approx \mathcal{L}_\infty + \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}.$$

The implication is that doubling compute typically requires roughly balanced doubling of N and D (Chinchilla recipe). Under-trained huge models (giant N , small D) underperform smaller models with more data.

5.5 Evaluation metrics (HW4 Q9)

Perplexity. For a held-out corpus,

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_\theta(w_t | w_{<t})\right) = \exp(\mathcal{L}_{\text{avg}}).$$

Interpret as the effective vocabulary size: a perplexity of 20 means the model is on average as confused as if it had to choose uniformly among 20 tokens.

Limitations: perplexity assumes the test text is the gold standard — the model is rewarded for matching the test distribution, not for being correct or useful.

BLEU (Bilingual Evaluation Understudy). For machine translation, counts overlapping n -grams between candidate and reference, with a brevity penalty:

$$\text{BLEU} = \text{BP} \exp\left(\sum_{n=1}^4 w_n \log p_n\right),$$

where p_n is modified n -gram precision and $\text{BP} = \min(1, e^{1-r/c})$ penalizes too-short translations (r = reference length, c = candidate length).

Limitations: surface-level n -gram match misses semantic equivalence (synonyms, paraphrases get 0 overlap); a single reference under-rewards legitimate alternative translations.

5.6 Post-training: fine-tuning

Full fine-tuning. Continue training all weights on a downstream task. Cost: storing a full copy of the model per task. Risk: *catastrophic forgetting* of the pre-trained capabilities if the fine-tuning set is small or out-of-distribution.

LoRA (Low-Rank Adaptation). Freeze the pre-trained weights $\mathbf{W}_0 \in \mathbb{R}^{d \times d}$ and learn a low-rank update:

$$\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times d}, \mathbf{B} \in \mathbb{R}^{d \times r}, r \ll d.$$

At training time you only update $\mathbf{A}, \mathbf{B}$ ($2dr$ parameters instead of d^2). At inference, you can either keep them as a side-car or merge into $\mathbf{W}$ once. Multiple LoRA adapters can be stored and swapped per task ($r = 8$ or $r = 16$ is typical).

Why LoRA is enough. The empirical observation: pre-trained weights live close to a low-rank manifold for many downstream tasks, so a rank- r correction is sufficient. LoRA dramatically reduces storage and training compute and avoids catastrophic forgetting (the base weights are untouched).

5.7 RLHF and instruction tuning

A pre-trained LLM models the distribution of internet text. That is not the same as “a helpful, harmless assistant.” Three-stage post-training:

1. **Supervised fine-tuning (SFT).** Fine-tune the model on high-quality (instruction, response) pairs written by humans.
2. **Reward model training.** Collect human preference data: given a prompt and two responses, humans label which is better. Train a reward model $r_\phi(\mathbf{x}, \mathbf{y})$ to predict the preference (typically a Bradley-Terry / logistic likelihood on pairwise comparisons).
3. **RL fine-tuning (PPO or similar).** Optimize the LLM policy π_θ to maximize expected reward r_ϕ , with a KL penalty to keep π_θ close to the SFT model:

$$\mathcal{J}(\theta) = \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \pi_\theta} [r_\phi(\mathbf{x}, \mathbf{y})] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}}).$$

The KL penalty is essential: without it the LLM can collapse onto a small set of high-reward outputs (*reward hacking*), losing diversity and language quality.

5.8 In-context learning and prompting

The most striking property of large LLMs: they can “learn” new tasks from examples in the prompt without any parameter updates.

Few-shot prompting.

Q: 2 + 3 = ?
A: 5
Q: 7 + 4 = ?
A:

The model emits “11.” Conditioning on examples shifts the model’s output distribution toward the inferred task. Behavior emerges only at sufficient model scale.

Chain-of-thought (CoT). For multi-step problems, prompting the model to “think step by step” before answering improves accuracy on arithmetic, logic, and commonsense tasks. Reflects the fact that the forward pass has fixed depth: serializing reasoning into tokens lets the model do unbounded computation by extending the context.

No gradients, only attention. ICL works by self-attention copying patterns from the prompt into the response. There is no weight update, so the “learning” is local to a single forward pass.

5.9 Retrieval-Augmented Generation (RAG)

Goal: ground LLM outputs in an external knowledge base.

1. Embed documents into a vector database (using a separate encoder).
2. At query time, embed the user’s query and retrieve the top- k similar documents.
3. Concatenate retrieved passages with the user query into the LLM’s prompt.

Why RAG: avoids re-training the LLM when knowledge changes; provides citation; reduces hallucination on factual questions. **Limits:** retrieval quality is the ceiling; LLM may still confabulate even when grounded.

5.10 Tool use and agents

The LLM can be given a list of available “tools” (functions) in the prompt. The model’s output may be either text or a structured tool call:

```
{"tool": "search", "args": {"query": "current weather in Paris"}}
```

The runtime executes the tool and pastes the result back into the context. This pattern is the basis for agents (Section 8).

5.11 Vision-language models (Dis 12 Q1)

Combine a vision encoder (e.g. CLIP, ViT) with an LLM.

Architecture. A vision encoder produces patch embeddings $\mathbf{H}_{\text{vis}} \in \mathbb{R}^{N \times d_{\text{vis}}}$. A projection layer $\mathbf{W}_P$ maps these to the LLM’s embedding space:

$$\mathbf{H}_{\text{proj}} = \mathbf{W}_P \mathbf{H}_{\text{vis}}, \quad \mathbf{W}_P \in \mathbb{R}^{d_{\text{txt}} \times d_{\text{vis}}}.$$

Dimensions: for $d_{\text{vis}}=1024$ and $d_{\text{txt}}=4096$, $\mathbf{W}_P$ has shape 4096×1024 . The projected image tokens are then prepended to text tokens and fed to the LLM.

Freezing strategy (Dis 12 Q1b). During SFT, freeze the vision encoder and the LLM, train only $\mathbf{W}_P$. Reasons:

1. Computational cost: $\mathbf{W}_P$ has $\sim$ millions of parameters; the LLM has billions. Training only $\mathbf{W}_P$ is orders of magnitude cheaper.
2. Avoids catastrophic forgetting: keeps both backbones’ pre-trained capabilities intact.

Image-token granularity (Dis 12 Q1c). Two choices:

- **Project the [CLS] token only:** cheap (one image token added to context) but loses spatial detail.
- **Project all patch tokens:** expensive (each image consumes ~ 256 context slots) but preserves location/structure information.

Why SFT is necessary even with matching dims (Dis 12 Q1d). The vision encoder and LLM are pre-trained independently. Their embedding spaces are not *aligned*: a “cat” in the vision encoder is some random direction that has no relation to the LLM’s “cat” embedding. SFT on image-text pairs teaches the projection layer to align them.

5.12 Pitfalls and quick reference

Pitfall 5.1 (Train/inference gap with teacher forcing). During training, the decoder always sees the ground-truth previous token. At inference it sees its own samples. Mismatched distributions can compound errors over long generations (*exposure bias*). Mitigations: scheduled sampling, beam search, RLHF.

Pitfall 5.2 (Catastrophic forgetting). Naive full fine-tuning on a small dataset can wipe out general capabilities. Use LoRA, replay (mix in pre-training data), or parameter-efficient methods to mitigate.

Pitfall 5.3 (Reward hacking). Without a KL penalty, RLHF lets the LLM exploit quirks in the reward model rather than improving on the actual task. The KL term tethers the policy to the SFT model.

Pitfall 5.4 (Perplexity vs. usefulness). A model with lower perplexity on web text is not necessarily a better assistant. Post-training (SFT + RLHF) can *worsen* perplexity while producing a much more useful model.

Cheat sheet.

- Pre-training loss: $\mathcal{L} = -\sum_t \log p_\theta(w_t \mid w_{<t})$.
- Perplexity: $\exp(\mathcal{L}_{\text{avg}})$.
- RMSNorm = LayerNorm without centering.
- LoRA: $\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}$, $\mathbf{B}\mathbf{A}$ rank $r \ll d$.
- RLHF objective: $\mathbb{E}[r_\phi(\mathbf{x}, \mathbf{y})] - \beta D_{\text{KL}}(\pi \parallel \pi_{\text{SFT}})$.
- In-context learning needs no gradients; only the forward pass.
- RAG: retrieve, concat, generate. Bound by retrieval quality.
- VLM projection: $\mathbf{W}_P$ shape $d_{\text{txt}} \times d_{\text{vis}}$; freeze the backbones during alignment.

6 Self-Supervised Learning & Generative Models

Self-supervised learning (SSL) trains representations using “free” supervision derived from the data itself, eliminating the bottleneck of human labels. This section covers (i) the catalogue of pretext tasks, (ii) contrastive learning and InfoNCE, (iii) the role of more negative samples, (iv) why latent-space contrast often beats reconstruction, (v) generative models (autoregressive vs. diffusion), and (vi) flow matching.

6.1 The SSL premise

Many domains have abundant unlabeled data (images on the web, raw text, genome sequences) but scarce labels. SSL constructs a *pretext task* whose supervision can be generated automatically from the data, then transfers the learned representation to a downstream task with few labels. Two families:

- **Generative pretext:** reconstruct missing or corrupted parts of the input. Loss: pixel-space MSE or likelihood.
- **Discriminative pretext:** classify a derived label (rotation angle, “same image as positive”). Loss: cross-entropy or contrastive.

6.2 Catalogue of pretext tasks (Dis 12 Q2a)

Method	Input	Pretext task	Type	Loss
Autoencoder	image	reconstruct input	generative	MSE
Context encoder	masked image	predict missing region	generative	MSE + adversarial
Image rotation	rotated image	predict rotation angle	discriminative	cross-entropy
SimCLR	two augmentations	same image or not	discriminative	contrastive (InfoNCE)

Why context-encoder reconstruction needs adversarial loss (Dis 12 Q2b). Pure L_2 reconstruction on the masked region:

$$\mathcal{L}_{\text{rec}} = \|M \odot (\mathbf{x} - \hat{\mathbf{x}})\|_2^2.$$

The minimizer of expected L_2 loss is the *conditional mean* of the data distribution given the visible context. When multiple completions are plausible (e.g. several sharp textures), the conditional mean averages them into a blurry compromise.

Adding an adversarial loss $\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{adv}}$ encourages the generator to pick a single sharp mode that the discriminator cannot distinguish from real. The adversarial term breaks the averaging symmetry.

6.3 Contrastive learning and InfoNCE (HW5 Q1)

Setup: an encoder $g_{\text{enc}} : \mathbb{R}^d \rightarrow \mathbb{R}^m$ produces representations $\mathbf{z}_i = g_{\text{enc}}(\mathbf{x}_i)$. For each anchor $\mathbf{x}_i$ define:

- One *positive* $\mathbf{z}_i^+$ (e.g. a different augmentation of $\mathbf{x}_i$, or a biological neighbor).
- $N-1$ *negatives* $\mathbf{z}_j^-$ from unrelated samples in the batch.

Score function $f(\mathbf{z}, \mathbf{c}_i) = \exp(\mathbf{z}^\top \mathbf{W} \mathbf{c}_i)$ with learnable $\mathbf{W}$ and a context vector $\mathbf{c}_i$ derived from $\mathbf{z}_i$.

InfoNCE loss.

$$\mathcal{L}_i = -\log \frac{f(\mathbf{z}_i^+, \mathbf{c}_i)}{f(\mathbf{z}_i^+, \mathbf{c}_i) + \sum_{j=1}^{N-1} f(\mathbf{z}_j^-, \mathbf{c}_i)}.$$

(a) Interpretation as N -way classification. The denominator runs over 1 positive and $N-1$ negatives. The loss is the negative log-likelihood of correctly classifying the positive among N candidates; the “classes” are the N candidate samples. Minimizing $\mathcal{L}_i$ makes the encoder push the positive’s score above all negatives’.

(b) Gradient w.r.t. scores $s_j = \mathbf{z}_j^\top \mathbf{W} \mathbf{c}_i$. Let $a_j = \exp(s_j) / \sum_k \exp(s_k)$ be the softmax probabilities. The InfoNCE loss is $\mathcal{L}_i = -s_+ + \log \sum_k \exp(s_k)$, so

$$\begin{aligned} \frac{\partial \mathcal{L}_i}{\partial s_+} &= -1 + a_+ = -(1 - a_+), \\ \frac{\partial \mathcal{L}_i}{\partial s_j^-} &= a_j. \end{aligned}$$

Interpretation:

- Positive: gradient is negative whenever $a_+ < 1$, so the update *increases* s_+ until the positive dominates.
- Negative: gradient is positive whenever $a_j > 0$, so the update *decreases* s_j^- , suppressing scores of unrelated samples.

The two forces together pull positives together in latent space and push negatives away — the contrastive principle.

(c) Why more negatives helps. As N grows, the denominator samples a tighter approximation to the full data distribution, so the classification problem becomes harder *and* more informative. In the limit $N \rightarrow \infty$, InfoNCE bounds the mutual information between anchor and positive:

$$I(\mathbf{z}_i^+; \mathbf{c}_i) \geq \log N - \mathcal{L}_i.$$

Higher N raises this lower bound, encouraging the encoder to learn representations that are maximally informative about the positive.

Tradeoff: larger N requires larger batches, more memory, more compute. SimCLR uses $N \approx 8192$ in 32 GPUs; MoCo uses a queue of features to maintain a large effective N without large batches.

(d) Contrastive vs. reconstruction. Pixel reconstruction forces the model to allocate capacity to noise and high-frequency texture that is irrelevant to semantics. Contrastive learning operates in a low-dimensional latent space, ignoring pixel-level details. The result: better downstream linear-probe accuracy with the same backbone.

(e) Transfer learning. After pre-training, freeze g_{enc} and train a small classifier on labeled data. This *linear probe* is the standard benchmark for SSL methods. It works because:

- The encoder has learned task-agnostic features over millions of unlabeled samples.
- The downstream classifier has few parameters, so it does not overfit even on small labeled sets.
- Catastrophic forgetting is impossible (encoder is frozen).

6.4 Generative SSL: AR vs. diffusion (Dis 12 Q3)

Both autoregressive (AR) and diffusion models perform a long sequence of sampling steps at inference. Why are they tractable to train?

Training efficiency. Both decouple the deep inference path from the training-step backpropagation:

- **AR model.** Trained to predict x_n given $x_{<n}$ for a single n . Backprop is through one prediction, not through the entire generation chain.
- **Diffusion model.** Trained to predict the noise ϵ injected at a single random timestep t . Backprop is through one denoising step, not through all T denoising steps.

The full deep computational graph (auto-regressive unroll, or diffusion’s reverse process) is constructed only at *inference* time, where no gradients are needed.

Frequency content during diffusion sampling. Empirically, the denoising process synthesizes:

- **High noise** ($t \approx T$): coarse global structure, low spatial frequencies (overall composition, large shapes, color regions).
- **Low noise** ($t \approx 0$): fine details, high spatial frequencies (textures, edges).

The model first decides “what the image is about” and then fills in local details.

6.5 Diffusion model theory (HW5 Q diffusion)

Setup. Define a forward noising process for $\sigma > 0$:

$$\mathbf{x}_\sigma = \mathbf{x}_0 + \sigma \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Theorem 6.1 (Conditional mean as L_2 minimizer (HW5 Q1a)). For random vectors $\mathbf{X}, \mathbf{Y}$ and any function f ,

$$\mathbb{E}[\|f(\mathbf{Y}) - \mathbf{X}\|^2] \geq \mathbb{E}[\|\mathbb{E}[\mathbf{X} | \mathbf{Y}] - \mathbf{X}\|^2].$$

The minimizer is $f^*(\mathbf{y}) = \mathbb{E}[\mathbf{X} | \mathbf{Y} = \mathbf{y}]$.

Sketch. Decompose $\mathbf{X} - f(\mathbf{Y}) = (\mathbf{X} - \mathbb{E}[\mathbf{X} | \mathbf{Y}]) + (\mathbb{E}[\mathbf{X} | \mathbf{Y}] - f(\mathbf{Y}))$. Cross-term has zero expectation by the tower property: $\mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X} | \mathbf{Y}])h(\mathbf{Y})] = \mathbb{E}[\mathbb{E}[\cdot | \mathbf{Y}]h(\mathbf{Y})] = 0$. Squared norm of a sum with vanishing cross-term equals sum of squared norms; the second is minimized by $f = \mathbb{E}[\mathbf{X} | \mathbf{Y}]$. $\square$

(b) Optimal denoiser as conditional expectation. The denoising loss

$$\mathcal{L}_\sigma(\boldsymbol{\epsilon}_\sigma) = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}}[\|\boldsymbol{\epsilon}_\sigma(\mathbf{x}_0 + \sigma \boldsymbol{\epsilon}) - \boldsymbol{\epsilon}\|^2]$$

is minimized at $\boldsymbol{\epsilon}_\sigma^*(\mathbf{x}_\sigma) = \mathbb{E}[\boldsymbol{\epsilon} | \mathbf{x}_\sigma]$. Using $\mathbf{x}_0 = \mathbf{x}_\sigma - \sigma \boldsymbol{\epsilon}$:

$$\boldsymbol{\epsilon}_\sigma^*(\mathbf{x}_\sigma) = \mathbb{E}\left[\frac{\mathbf{x}_\sigma - \mathbf{x}_0}{\sigma} \middle| \mathbf{x}_\sigma\right] = \frac{\mathbf{x}_\sigma - \mathbb{E}[\mathbf{x}_0 | \mathbf{x}_\sigma]}{\sigma}.$$

Equivalently, by Tweedie's formula, in terms of the data density p_σ ,

$$\epsilon_\sigma^*(\mathbf{x}_\sigma) = -\sigma \nabla_{\mathbf{x}_\sigma} \log p_\sigma(\mathbf{x}_\sigma).$$

The optimal denoiser is the (negative scaled) score of the noised distribution. This is the bridge between denoising and *score-based* generative modeling: train a network to predict noise, and you have trained a score estimator.

(c) Continuity equation (HW5 Q1c). For a continuous noise schedule $\sigma(t)$ with corresponding density p_t on $\mathbf{x}_t = \mathbf{x}_0 + \sigma(t)\epsilon$, define the velocity field

$$\mathbf{v}(\mathbf{x}, t) = \mathbb{E} \left[\frac{\partial}{\partial t} \sigma(t) \epsilon \mid \mathbf{x}_t = \mathbf{x} \right].$$

Then p_t satisfies the *continuity equation*

$$\partial_t p_t(\mathbf{x}) + \nabla \cdot (\mathbf{v}(\mathbf{x}, t) p_t(\mathbf{x})) = 0.$$

Sketch of why: if every particle's position evolves as $d\mathbf{x}/dt = \mathbf{v}$, the local mass conservation law is precisely the continuity equation. The diffusion ODE/SDE that one integrates at inference is the deterministic flow defined by this $\mathbf{v}$ (probability flow ODE).

6.6 Flow matching (HW5 flow-matching problem)

A simpler alternative to score-matching diffusion: directly learn a velocity field that transports a known noise distribution to the data distribution.

Linear interpolation path. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$ (noise) and $\mathbf{x}_1 \sim p_{\text{data}}$ independently, then define

$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \quad t \in [0, 1].$$

The endpoints satisfy $\mathbf{x}_0$ at $t = 0$ and $\mathbf{x}_1$ at $t = 1$. The ground-truth velocity along this path is constant:

$$\dot{\mathbf{x}}_t = \frac{\partial \mathbf{x}_t}{\partial t} = \mathbf{x}_1 - \mathbf{x}_0.$$

General interpolation. Replace $t, 1-t$ with differentiable functions $\alpha(t), \beta(t)$ satisfying

$$\alpha(0) = 0, \beta(0) = 1, \alpha(1) = 1, \beta(1) = 0.$$

Then $\mathbf{x}_t = \alpha(t)\mathbf{x}_1 + \beta(t)\mathbf{x}_0$ and

$$\dot{\mathbf{x}}_t = \alpha'(t)\mathbf{x}_1 + \beta'(t)\mathbf{x}_0.$$

For the linear case $\alpha(t) = t, \beta(t) = 1-t$ this reduces to $\dot{\mathbf{x}}_t = \mathbf{x}_1 - \mathbf{x}_0$ as expected.

Single-function parameterization. Let $\mathbf{x}_t = h(t)\mathbf{x}_1 + (1-h(t))\mathbf{x}_0$ for monotone $h(0) = 0, h(1) = 1$. Then

$$\dot{\mathbf{x}}_t = h'(t)(\mathbf{x}_1 - \mathbf{x}_0).$$

$h(t)$ controls the *schedule* along the path:

- $h(t) = t$: constant speed.
- $h(t) = t^2$: $h'(t) = 2t$, so motion is slow near $t = 0$ and fast near $t = 1$.
- $h(t) = \sin(\pi t/2)$: $h'(t) = \frac{\pi}{2} \cos(\pi t/2)$, so motion is fast near $t = 0$ and slow near $t = 1$.

Training objective. Train a network $\mathbf{v}_\theta(\mathbf{x}, t)$ to match the ground-truth velocity:

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} \left[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \dot{\mathbf{x}}_t\|^2 \right].$$

At inference, integrate the ODE $d\mathbf{x}/dt = \mathbf{v}_\theta(\mathbf{x}, t)$ from $t = 0, \mathbf{x} = \mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$ to $t = 1$ to obtain a sample.

6.7 Pitfalls and quick reference

Pitfall 6.2 (Mode collapse without negatives). Contrastive learning with positives only (no negatives) admits the trivial constant solution: all $\mathbf{z} = \mathbf{c}$ gives perfect score $f(\mathbf{z}_i^+, \mathbf{c}_i) = e^{\|\mathbf{c}\|_W^2}$. Negatives are necessary to push the encoder away from this collapse. (Variants like BYOL avoid explicit negatives via stop-gradient and momentum encoders.)

Pitfall 6.3 (Reconstruction loss averages over modes). L_2 reconstruction targets the conditional mean. When multiple completions are plausible, the mean is blurry. Use adversarial, perceptual, or sampling-based losses for sharp outputs.

Pitfall 6.4 (Direction of noise in diffusion). The forward process *adds* noise; the reverse process *removes* it. The ODE used at inference time integrates the velocity in the direction of decreasing noise. Sign mistakes here flip generation into denoising-into-noise, which is uninformative.

Pitfall 6.5 (Score vs. noise prediction parameterization). $\epsilon_\sigma^* = -\sigma \nabla \log p_\sigma$. Two equivalent parameterizations: predict ϵ (noise) or predict $\nabla \log p_\sigma$ (score). They differ only by a $-\sigma$ factor.

Cheat sheet.

- InfoNCE loss: $-\log[f(\mathbf{z}^+, \mathbf{c}) / (f(\mathbf{z}^+, \mathbf{c}) + \sum f(\mathbf{z}^-, \mathbf{c}))]$.
- More negatives $\Rightarrow$ tighter MI lower bound.
- Conditional mean minimizes expected L_2 .
- Optimal denoiser: $\epsilon_\sigma^* = -\sigma \nabla \log p_\sigma$.
- Diffusion training: predict noise at one random timestep.
- Flow matching: $\mathbf{x}_t = h(t)\mathbf{x}_1 + (1 - h(t))\mathbf{x}_0$, $\dot{\mathbf{x}}_t = h'(t)(\mathbf{x}_1 - \mathbf{x}_0)$.
- Continuity equation: $\partial_t p + \nabla \cdot (\mathbf{v}p) = 0$.
- Coarse-to-fine generation: low frequencies appear first in diffusion sampling.

7 AI for Protein Engineering

A short tour of where ML lands in computational biology, framed as *prediction tasks* (sequence/structure/function) and *design tasks* (generate sequences with desired function). The exam-relevant takeaways are: (i) the three families of conditioning strategies for design, (ii) why diffusion/flow are particularly well-suited, and (iii) the extrapolation/distribution-shift trap that makes naive design fail.

7.1 Why proteins are hard

A protein of length L over $A = 20$ amino acids inhabits a discrete space of size A^L :

- Antibody $L \sim 100$: $20^{100} \approx 10^{130}$, far more than $\sim 10^{80}$ atoms in the universe.
- No gradients exist on the discrete space directly — standard diffusion/score methods do not apply out of the box.
- The fitness landscape is highly rugged: most sequences don't fold; most folded sequences don't have the desired function.

Three historical approaches:

1. **Physics-based** energy functions (Rosetta, etc.): no training data needed, but accuracy is limited.
2. **Directed evolution** in the wet lab: cycles of mutate, screen, select. Works but slow and limited to small distances from the starting sequence.
3. **Machine learning**: train on natural sequences (millions available), use the resulting model for prediction or as a prior for design.

7.2 Structure prediction: AlphaFold2

Given a sequence, predict the 3D structure (atom coordinates). AlphaFold2 (2020) achieved near-experimental accuracy on benchmark targets.

Architectural ingredients.

- Multi-sequence alignments (MSAs) of evolutionary related sequences as input.
- Co-evolution / contact prediction (deep features capturing which residues mutate together $\Rightarrow$ likely physically near in 3D).
- A rotation-equivariant attention module (the “structure module”) that respects 3D geometric invariances.
- Final refinement with classical physics-based energy functions (raw NN output may violate physical constraints).

What it didn't solve. Structure prediction is one piece; *design* requires structure $\rightarrow$ sequence (inverse folding) and, critically, function prediction. We typically don't even know which structure we want: we want a function, and the structure-function map is not solved.

7.3 Design = conditional generation

Setup: have an unconditional sequence model $p_\theta(\mathbf{x})$ trained on natural proteins (e.g., ESM3, ProteinMPNN). Want a conditional model $p_\theta(\mathbf{x} \mid y)$, where y is a desired property (binding affinity, expression, EC number, etc.). Three strategies:

1. **Train conditional from scratch.** $p_\theta(\mathbf{x} \mid y)$ directly. Requires paired $(\mathbf{x}, y)$ data, which is usually scarce.
2. **Post-hoc update of unconditional model.** Examples: CbAS, DPO. Use a property-prediction model $p(y \mid \mathbf{x})$ to reweight or fine-tune the unconditional model.
3. **On-the-fly guidance.** Freeze the unconditional generative model; steer generation at inference time using a separate property predictor (e.g., classifier guidance for diffusion).

All three are statistically principled if you apply Bayes’ rule:

$$p(\mathbf{x} \mid y) \propto p(y \mid \mathbf{x}) p(\mathbf{x}).$$

7.4 Why diffusion/flow are well-suited (continuous)

Diffusion models train networks to predict scores $\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x})$ directly. Bayes’ rule in score form is convenient:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid y) = \nabla_{\mathbf{x}} \log p(y \mid \mathbf{x}) + \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

The intractable normalizing constant of the conditional density vanishes under the gradient. This is *classifier guidance*: at each denoising step, add a step in the direction $\nabla \log p(y \mid \mathbf{x})$ from a property predictor.

7.5 Discrete-space challenges

Proteins are discrete. Two issues:

- Standard diffusion/score does not apply: $\nabla_{\mathbf{x}} \log p$ is ill-defined for discrete $\mathbf{x}$.
- Standard guidance via $\nabla_{\mathbf{x}} \log p(y \mid \mathbf{x})$ similarly ill-defined.

Mitigations covered in lecture:

1. Relax discrete tokens to a continuous embedding, generate, then “snap back” to the nearest discrete sequence.
2. Diffusion on the multinomial simplex (each position is a probability vector over amino acids).
3. Continuous-time Markov chains (CTMCs): jumps between discrete states with controllable rate matrices, allowing both diffusion and guidance on discrete spaces.

7.6 Inductive biases for proteins

A few examples of protein-specific architectural choices:

- Rotation/translation equivariance for structure modules (AlphaFold).
- Pairwise-residue “contact” features as graph edges.
- Permutation symmetry over interchangeable chains in protein complexes.
- Spectral / Fourier representations of fitness landscapes (epistasis decomposition: any fitness function on length- L sequences is exactly a sum of single-site, pairwise, and higher-order epistatic terms up to order L).

7.7 The pathology of naive ML-based design

The cautionary tale (cf. “banana design” analogy in lecture):

If you maximize a property predictor $\hat{f}(\mathbf{x})$ over the space of sequences, you find sequences that fool the predictor into reporting high $\hat{f}$ *without* actually having the property. The “optimal” designed

sequence ends up in a region of sequence space the predictor has never seen, where its predictions are unreliable.

Pitfall 7.1 (Distribution shift in design). A predictor $\hat{f}$ trained on natural proteins is trustworthy on sequences *near* the training distribution. Design pushes generation *away* from natural sequences (otherwise we’d just use existing ones). Naive design without uncertainty/extrapolation control is the adversary that defeats the predictor.

Mitigations covered.

- Design over *distributions* of sequences rather than single points (CbAS, DbAS).
- Penalize departure from the training distribution (by KL or unconditional $p_\theta(\mathbf{x})$ score).
- Estimate epistemic uncertainty (e.g. Bayesian NNs, conformal prediction) and avoid regions where the predictor is uncertain.

7.8 Pitfalls and quick reference

Pitfall 7.2 (Structure $\neq$ function). AlphaFold predicts structure from sequence. It does not tell you whether a given structure has the function you want. The structure $\rightarrow$ function map is the open problem.

Pitfall 7.3 (Discrete diffusion sign errors). Standard score-based methods break for discrete $\mathbf{x}$. Use either relaxation, simplex diffusion, or CTMC frameworks — not vanilla score-matching.

Cheat sheet.

- Sequence space size: A^L , $A = 20$ for amino acids.
- AlphaFold2: MSA features + co-evolution + equivariant attention + physics-based refinement.
- Three conditioning strategies: bake-in / post-hoc update / on-the-fly guidance.
- Score form of Bayes: $\nabla \log p(\mathbf{x} | y) = \nabla \log p(y | \mathbf{x}) + \nabla \log p(\mathbf{x})$.
- Discrete spaces: relax-and-snap, simplex diffusion, or CTMCs.
- Pathology: optimizing a predictor over sequence space hits out-of-distribution regions where the predictor is unreliable.

8 LLMs, Agents, & Environments

An *agent* is an LLM whose control flow is determined by its own outputs in a loop with an environment. This section places agents in the context of the evolution of LLM systems, defines what makes a system “agentic”, describes the post-training pipeline (environments, verifiers, RL) that shifts effort from data to environments, and covers the prompt-optimization mechanic GEPA from lecture.

8.1 Taxonomy: LLM, tool-use, RAG, workflow, agent

A ladder of increasing autonomy:

- **Plain LLM.** Tokens in, tokens out. No external interaction.
- **LLM with tool use.** The LLM emits a tool call (e.g. a shell command). A runtime executes it and pastes the result back in.
- **Retrieval-Augmented Generation (RAG).** A retrieval step finds relevant documents, the LLM answers given them. Hard-coded workflow with a retrieval tool call.
- **Workflow.** Multi-step pipeline with branching: “if LLM says X, run Python code 1; if Y, run code 2.” Control flow is partially decided by the LLM but is still bounded by hand-coded topology.
- **Agent.** The LLM decides what to do next, indefinitely. It can call tools, read state, plan, retry. Loop continues until the agent decides it’s done.

ReACT loop. The canonical agent inner loop:

1. Read state of the world (e.g., terminal output, files, prior tool results).
2. LLM emits a thought + an action (a tool call or final answer).
3. Runtime executes the action, returning a new state.
4. Repeat.

8.2 Three eras of LLM systems

1. **Embeddings era (2018–2020):** BERT, GPT-2 used as feature extractors. Pre-train, freeze, fine-tune a head.
2. **Assistants era (2022–2024):** ChatGPT-style. Pre-train + RLHF. Single conversational turn or short multi-turn.
3. **Tools era (2024–2025):** LangChain, AutoGen, RAG, workflows. Single LLM call $\Rightarrow$ multiple LLM calls.
4. **Autonomous agents (2026–):** Claude Code, Codex. LLM owns control flow over many steps.

8.3 The long-horizon problem

Agents become unstable after roughly 3–5 steps. Failure modes:

- **Goal drift:** agent forgets its top-level objective and starts solving a side task.
- **Tool-call hallucination:** agent calls tools or arguments that don’t exist.
- **Context bloat:** useful context is buried under tool output noise.
- **Reward hacking:** if the agent is being trained, it exploits the reward function instead of solving the task.

These failures don’t show up in single-turn LLM benchmarks; they require multi-turn agent evals.

8.4 Environments and Terminal-Bench

For real-world agents (especially code agents) the bottleneck has shifted: the dominant cost is no longer collecting Q&A pairs but constructing rich, verifiable *environments* for the agent to operate in. Terminal-Bench (Schmidt et al.) defines an environment as a triple:

1. **Task description.** Natural-language statement of what the agent should accomplish, e.g. “my python installation is broken, install pip and re-install package *X*.”
2. **Environment.** A Dockerfile that sets up an executable context (an OS, an installed Python, deliberately broken state, etc.).
3. **Verifier.** An automated test that checks whether the task was completed successfully (e.g. “does `pip install foo` now work without errors?”).

The verifier is the critical piece. Without an automatic verifier you cannot do RL fine-tuning, you cannot benchmark, and you cannot iterate quickly. Verifiers turn “did the agent solve this” into a binary reward signal.

LM eval vs. agent eval.

LM eval	Agent eval
Data = (input, gold output) pairs.	Data = environments.
Compare model output to gold.	Compare model <i>actions</i> or final state.
Single turn.	Multi-turn, interactive.
Well-defined right answer.	Often ambiguous; partial credit.

8.5 Skills: hierarchical capability composition

A practical pattern for giving agents large libraries of capabilities:

- Skills are organized in a folder hierarchy, with each skill a small bundle (instructions + examples + helper scripts).
- The agent navigates the hierarchy at runtime to find the relevant skill, instead of stuffing all instructions into a single mega-prompt.
- This works because LLMs are unreasonably good at filesystem navigation: hierarchical organization, `grep`, and unix shell composition all play to LLM strengths.

This is the same idea as RAG (retrieve relevant content into context) but for capabilities rather than facts.

8.6 GEPA: prompt optimization via natural-language reflection

When you have an environment with a verifier, the prompt itself becomes a tunable parameter. *GEPA* (Genetic-Pareto evolution of prompts) is the optimizer covered in lecture:

1. Run the agent with a current prompt on a batch of tasks; record the verifier outcomes.
2. For each failure, ask an LLM to “reflect” on what went wrong in natural language and propose a better prompt.
3. Combine reflections via a genetic-style crossover/mutation operator over prompts.
4. Score candidate prompts by their verifier outcomes; keep the Pareto frontier.
5. Repeat.

Compared to gradient-based optimization, GEPA’s “gradient” is a language-model-generated suggestion in natural language. This is enabled by the verifier: without binary success signals there’s nothing for the reflection step to point at.

8.7 Pitfalls and quick reference

Pitfall 8.1 (“Multi-agent” is “multi-prompt”). “Multi-agent systems” usually means multiple LLM calls with different system prompts. There is no fundamental new mechanism; just a different factoring of work across prompts. Don’t be impressed by the label.

Pitfall 8.2 (Unverifiable tasks defeat agent training). Without an automated verifier, RL fine-tuning is impossible and benchmarking is anecdotal. Many “agent demos” work in cherry-picked cases but cannot be improved systematically.

Pitfall 8.3 (Long-horizon failure compounds). A 90%-per-step success rate over 10 steps gives only $0.9^{10} \approx 0.35$ end-to-end success. Agents need either much higher per-step reliability or robust recovery from intermediate errors.

Cheat sheet.

- Agent vs. workflow: agent’s control flow is open-ended; workflow’s topology is hand-coded.
- ReACT loop: state $\rightarrow$ thought + action $\rightarrow$ tool execute $\rightarrow$ new state.
- Environment = (task description, executable environment, verifier).
- Terminal-Bench: framework + 80 expert-level tasks for code agents.
- Skills as folder hierarchy: agents traverse with shell commands.
- GEPA: natural-language reflection on verifier failures, evolves prompts.

9 Information Theory & the Information Bottleneck

Information theory provides language for talking about “how much information one variable carries about another.” Lecture 16 introduces the basics; HW3 Q4 builds on them to characterize neural-network training through the *information bottleneck* (IB) framework. This section collects definitions, the key inequalities (Gibbs’, max-entropy, data processing), the rate-distortion vs. IB framing, and the two-phase picture of SGD as fit-then-compress.

9.1 Shannon information and entropy

Use $\log = \log_2$ throughout (units: bits).

Definition 9.1 (Shannon information content). For an outcome x with probability $p(x)$,

$$h(x) = \log \frac{1}{p(x)}.$$

Rare outcomes carry more bits of information.

Definition 9.2 (Entropy). For a discrete random variable X over alphabet $\mathcal{A}_X$,

$$H(X) = \mathbb{E}_{X \sim p}[h(X)] = \sum_{x \in \mathcal{A}_X} p(x) \log \frac{1}{p(x)}.$$

$H(X) \geq 0$, with equality iff p is a point mass.

Joint and conditional entropy.

$$\begin{aligned} H(X, Y) &= \sum_{x, y} p(x, y) \log \frac{1}{p(x, y)}, \\ H(X | Y) &= \sum_{x, y} p(x, y) \log \frac{1}{p(x | y)} = H(X, Y) - H(Y). \end{aligned}$$

9.2 KL divergence and Gibbs’ inequality

Definition 9.3 (KL divergence). For two distributions p, q over the same alphabet,

$$D_{\text{KL}}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Asymmetric: $D_{\text{KL}}(p \| q) \neq D_{\text{KL}}(q \| p)$ in general.

Theorem 9.4 (Gibbs’ inequality). $D_{\text{KL}}(p \| q) \geq 0$, with equality iff $p = q$.

Proof. Apply Jensen’s inequality to the convex function $-\log$:

$$D_{\text{KL}}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = -\mathbb{E}_p \left[\log \frac{q(X)}{p(X)} \right] \geq -\log \mathbb{E}_p \left[\frac{q(X)}{p(X)} \right] = -\log 1 = 0.$$

Equality holds iff $q(x)/p(x)$ is constant, i.e. $p = q$. □

9.3 Maximum entropy and Kraft's bound (HW3 Q4b)

Theorem 9.5 ($H(X) \leq \log m$). For X over alphabet of size m ,

$$H(X) \leq \log m,$$

with equality iff X is uniform.

Proof. Apply Gibbs' inequality with $q(x) = 1/m$ uniform:

$$0 \leq D_{\text{KL}}(p||q) = \sum p(x) \log \frac{p(x)}{1/m} = -H(X) + \log m.$$

Hence $H(X) \leq \log m$, equality iff $p(x) = 1/m$. □

Kraft inequality $\Rightarrow$ codeword bound. For any uniquely decodable binary code with codeword lengths $\ell(x)$,

$$\sum_x 2^{-\ell(x)} \leq 1.$$

Define $q(x) = 2^{-\ell(x)}/C$ where $C = \sum 2^{-\ell(x)} \leq 1$. Then by Gibbs',

$$0 \leq D_{\text{KL}}(p||q) = \sum p(x) \log \frac{p(x)}{2^{-\ell(x)}/C} = -H(X) + \mathbb{E}_p[\ell(X)] + \log C.$$

Since $\log C \leq 0$, we get

$$\boxed{\mathbb{E}_p[\ell(X)] \geq H(X).}$$

The expected codeword length is bounded below by the entropy — the Source Coding Theorem.

9.4 Mutual information

Definition 9.6 (Mutual information).

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X) = D_{\text{KL}}(p(x, y) || p(x)p(y)).$$

$I(X; Y) \geq 0$ (since it's a KL); $I(X; Y) = 0$ iff $X \perp Y$. Symmetric: $I(X; Y) = I(Y; X)$.

Conditional mutual information.

$$I(X; Y | Z) = H(X | Z) - H(X | Y, Z).$$

Invariance under invertible maps. If ϕ is invertible, $I(X; \phi(Y)) = I(X; Y)$ and $H(\phi(Y)) = H(Y)$.

Data Processing Inequality (DPI). If $X \rightarrow Y \rightarrow Z$ form a Markov chain (all info from X to Z flows through Y), then

$$I(X; Z) \leq I(X; Y).$$

Processing can never *add* information about X , only preserve or lose it.

9.5 Rate-distortion theory

A short pre-cursor to the IB framework. In lossy compression, X is encoded into $\tilde{X}$ with distortion measured by $d(x, \tilde{x})$ (e.g. MSE for images). The minimum bit rate to achieve average distortion $\leq D$ is

$$R(D) = \min_{p(\tilde{x}|x): \mathbb{E}[d(X, \tilde{X})] \leq D} I(X; \tilde{X}).$$

Catch: the distortion function d must be chosen *a priori*. For most domains we don't know what "good distortion" is.

9.6 The Information Bottleneck (HW3 Q4c-d)

The IB framework sidesteps the distortion question. Replace $d(x, \tilde{x})$ with a *relevance variable* Y that says what should be preserved. The Markov structure

$$Y \leftrightarrow X \rightarrow \tilde{X}$$

captures: Y and X are correlated arbitrarily; $\tilde{X}$ is computed from X alone. Conditional independence: $\tilde{X} \perp Y \mid X$. By DPI, $I(\tilde{X}; Y) \leq I(X; Y)$ — compression can only lose information about Y .

IB objective.

$$\mathcal{L}[p(\tilde{x} \mid x)] = I(X; \tilde{X}) - \beta I(\tilde{X}; Y),$$

where $\beta > 0$ controls the compression-prediction tradeoff:

- $\beta \rightarrow 0$: minimize $I(X; \tilde{X})$ alone $\Rightarrow$ trivial collapse $\tilde{X} = \text{const.}$
- $\beta \rightarrow \infty$: maximize $I(\tilde{X}; Y)$ alone $\Rightarrow \tilde{X} = X$ (no compression).
- Intermediate β : keep only the bits of X that matter for Y .

9.7 The IB identity (HW3 Q4d-i)

A surprising identity that ties IB to a clean "compression-loss" picture. For any deterministic $X \rightarrow \tilde{X} = \tilde{x}(X)$,

$$I(X; Y) - I(\tilde{X}; Y) = \mathbb{E}_p \left[D_{\text{KL}} \left(p(y \mid X) \parallel p(y \mid \tilde{X}) \right) \right].$$

Sketch of proof.

$$I(X; Y) - I(\tilde{X}; Y) = H(Y \mid \tilde{X}) - H(Y \mid X).$$

Expand both conditional entropies, use $p(\tilde{x})p(y \mid \tilde{x}) = \sum_{x \in \tilde{x}} p(x)p(y \mid x)$ (determinism: knowing $\tilde{x}$ gives the set of x 's mapping to it), and compare term by term. The difference simplifies to the expected KL between $p(y \mid X)$ and the coarsened $p(y \mid \tilde{X})$.

Why this matters. Maximizing $I(\tilde{X}; Y)$ (the IB objective's prediction term) is *equivalent* to minimizing $\mathbb{E}_p[D_{\text{KL}}(p(y \mid X) \parallel p(y \mid \tilde{X}))]$ — the average KL between the true posterior over Y and the posterior implied by the compressed $\tilde{X}$.

Resolution of the rate-distortion catch: the IB framework discovers the right distortion function automatically. *KL between posteriors over Y* plays the role of $d(x, \tilde{x})$, and nobody had to choose it — it emerged from specifying Y .

9.8 The information plane and SGD’s two phases

For a deep network with input X , hidden layers $T_1, \dots, T_K$, and output $\hat{Y}$, every layer participates in the Markov chain

$$Y \leftrightarrow X \rightarrow T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_K \rightarrow \hat{Y}.$$

Plot each layer at its coordinates $(I(X; T_i), I(T_i; Y))$ in the *information plane*.

Two phases of training (Schwartz-Ziv & Tishby 2017).

1. **Fitting / drift.** All layers rapidly increase $I(T_i; Y)$ — the network learns to predict labels. SGD updates have high signal-to-noise ratio (consistent direction across batches).
2. **Compression / diffusion.** Layers slowly decrease $I(X; T_i)$ — the network discards input details irrelevant to Y . SGD updates become noisy with small mean (low SNR), so weights evolve like a random walk. Stochastic noise is what drives compression.

Why deeper networks compress more. Each layer can only lose information (DPI), so deeper stacks have more opportunity for compression. This is consistent with the empirical observation that depth helps even for representational tasks where one hidden layer would in principle suffice.

9.9 Why noise destroys irrelevant information first (HW3 Q4g)

Consider a binary hidden representation $T \in \{0, 1\}$ with $P(T = 1) = \pi$ and its noisy copy T' obtained by flipping with probability ϵ .

Theorem 9.7. $I(X; T') = H(T') - H(\epsilon)$ where $H(\epsilon) = -\epsilon \log \epsilon - (1 - \epsilon) \log(1 - \epsilon)$.

Reasoning. As ϵ rises from 0 to 1/2, $H(\epsilon)$ rises from 0 to 1, so $I(X; T')$ falls. At $\epsilon = 1/2$, T' is independent of T (a coin flip), so $I(X; T') = 0$ — the representation has lost all information about X .

Why irrelevant info is more fragile.

- **Coarse decisions** (e.g. “which side of the decision boundary is X on?”) are encoded in robust, large-margin features. Small weight noise rarely flips them.
- **Fine details** (e.g. exact distance to the decision boundary) are encoded in fine-grained weight values. Small weight noise easily perturbs them.

Adding noise to representations therefore preferentially destroys the fine-grained, label-irrelevant detail while preserving the coarse, label-relevant structure — which is exactly what we want from compression.

9.10 Pitfalls and quick reference

Pitfall 9.8 (KL is not symmetric). $D_{\text{KL}}(p||q)$ and $D_{\text{KL}}(q||p)$ measure different things and have different optima. Forward KL (p on outside) is mode-covering; reverse KL (q on outside) is mode-seeking. Always check which direction is in use.

Pitfall 9.9 (Mutual information is invariant under invertible maps). A common error: thinking that “a fancier feature transform” increases $I(X; \phi(X))$. It doesn’t — $I(X; \phi(X)) = H(X)$ for any invertible ϕ . Compression requires *many-to-one* maps (e.g., quantization, noise, ReLU saturation).

Pitfall 9.10 (DPI is one-way). $X \rightarrow Y \rightarrow Z$ implies $I(X; Z) \leq I(X; Y)$. The reverse is not implied; processing can only lose information, never add it.

Cheat sheet.

- Entropy: $H(X) = -\sum p(x) \log p(x) \leq \log |\mathcal{A}_X|$.
- KL: $D_{\text{KL}}(p \| q) = \sum p \log(p/q) \geq 0$ (Gibbs).
- Source coding: $\mathbb{E}[\ell(X)] \geq H(X)$.
- Mutual info: $I(X; Y) = H(X) - H(X | Y)$.
- DPI: $X \rightarrow Y \rightarrow Z \Rightarrow I(X; Z) \leq I(X; Y)$.
- Rate-distortion: $R(D) = \min I(X; \tilde{X})$ subject to distortion bound.
- IB: minimize $I(X; \tilde{X}) - \beta I(\tilde{X}; Y)$.
- IB identity: $I(X; Y) - I(\tilde{X}; Y) = \mathbb{E}_p[D_{\text{KL}}(p(y | X) \| p(y | \tilde{X}))]$.
- SGD phases: fit (rise of $I(T; Y)$, large gradients) then compress (fall of $I(X; T)$, noisy small gradients).
- Noise destroys high-precision info preferentially over coarse-margin info.

Appendix A: Worked Problems

This appendix fully solves every problem from discussions 8–12 and the written portions of HW3–5. Use these as practice: read the problem statement, attempt it without looking, then check your work.

Discussion 8 Q1: Gradient Descent Convergence

Setup. $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is smooth, μ -strongly convex, with L -Lipschitz gradients ($\mu I \preceq \nabla^2 f \preceq LI$). Iterate $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$.

(a) $\mathbf{x}^*$ is the unique local minimizer. Suppose $\mathbf{x}' \neq \mathbf{x}^*$ is also a local min. Define $g(t) = f(\mathbf{x}^* + t(\mathbf{x}' - \mathbf{x}^*))$ on $[0, 1]$. Then $g'(t) = \nabla f(\mathbf{x}^* + t(\mathbf{x}' - \mathbf{x}^*))^\top (\mathbf{x}' - \mathbf{x}^*)$, $g'(0) = g'(1) = 0$, and $g''(t) = (\mathbf{x}' - \mathbf{x}^*)^\top \nabla^2 f(\cdots)(\mathbf{x}' - \mathbf{x}^*) \geq \mu \|\mathbf{x}' - \mathbf{x}^*\|^2 > 0$. But then $g'(1) - g'(0) = \int_0^1 g''(t) dt > 0$, contradicting $g'(0) = g'(1) = 0$. So $\mathbf{x}^*$ is unique.

(b) Convergence for $0 < \alpha < 2/L$. Let $T(\mathbf{x}) = \mathbf{x} - \alpha \nabla f(\mathbf{x})$. Its Jacobian is $J_T(\mathbf{x}) = I - \alpha \nabla^2 f(\mathbf{x})$, with eigenvalues in $[1 - \alpha L, 1 - \alpha \mu]$. So $\|J_T\|_2 \leq M = \max\{|1 - \alpha \mu|, |1 - \alpha L|\} < 1$ under $0 < \alpha < 2/L$. By the mean value inequality, $\|T(\mathbf{x}_2) - T(\mathbf{x}_1)\| \leq M \|\mathbf{x}_2 - \mathbf{x}_1\|$. Since $T(\mathbf{x}^*) = \mathbf{x}^*$, induction gives $\|\mathbf{x}_k - \mathbf{x}^*\| \leq M^k \|\mathbf{x}_0 - \mathbf{x}^*\| \rightarrow 0$.

(c) Iteration complexity. From $M^k \|\mathbf{x}_0 - \mathbf{x}^*\| \leq \epsilon$, $k \geq \log(\|\mathbf{x}_0 - \mathbf{x}^*\|/\epsilon) / \log(1/M) = \mathcal{O}(\log(1/\epsilon))$.

(d) Optimal step size. $M(\alpha) = \max\{|1 - \alpha \mu|, |1 - \alpha L|\}$ is minimized when both arms are equal, i.e. $1 - \alpha \mu = -(1 - \alpha L)$, giving $\alpha^* = 2/(L + \mu)$ and $M^* = (\kappa - 1)/(\kappa + 1)$, $\kappa = L/\mu$.

Discussion 8 Q2: Universal Approximation via ReLU

Goal. For continuous $f : [0, 1] \rightarrow \mathbb{R}$ and $\epsilon > 0$, construct $F(x) = b_0 + \sum_{i=1}^N a_i \text{ReLU}(w_i x + b_i)$ with $\sup_{[0,1]} |f - F| < \epsilon$.

Construction. Uniform continuity gives $\delta > 0$ with $|x - y| < \delta \Rightarrow |f(x) - f(y)| < \epsilon$. Partition $0 = x_0 < x_1 < \cdots < x_n = 1$ with $x_k - x_{k-1} < \delta$. The piecewise-linear interpolant $p(x) = f(x_{k-1}) + m_k(x - x_{k-1})$ on $[x_{k-1}, x_k]$, with slopes $m_k = (f(x_k) - f(x_{k-1})) / (x_k - x_{k-1})$, satisfies $|f - p| < \epsilon$ on each interval. Now write p as ReLU sum:

$$F(x) = C + m_1 \text{ReLU}(x) + \sum_{k=1}^{n-1} (m_{k+1} - m_k) \text{ReLU}(x - x_k),$$

with C chosen so $F(x_0) = f(x_0)$. Then $F' = p'$ on each open interval, so $F \equiv p$, giving $\sup |f - F| < \epsilon$.

Discussion 9 Q1: Backprop on $f(x, y, z) = (x + y)z$

(a) Decompose: $q = x + y$, $f = qz$. **(b)** See the staged graph in Section 2. **(c) Forward pass:** compute q , then f . **Backward pass:** $\partial f / \partial z = q$, $\partial f / \partial q = z$, then $\partial f / \partial x = (\partial f / \partial q) \cdot 1 = z$ and similarly $\partial f / \partial y = z$. **(d) Numerical pass at $(x, y, z) = (-2, 5, -4)$:** Forward: $q = 3$, $f = -12$. Backward: $\bar{f} = 1$, $\bar{q} = z = -4$, $\bar{z} = q = 3$, $\bar{x} = \bar{y} = \bar{q} \cdot 1 = -4$.

Discussion 9 Q2: 1D and 2D Convolution

(a) **1D, $D = 9$, $K = 3$, no pad, stride 1.** Output dim $9 - 3 + 1 = 7$. First entry $k_1x_1 + k_2x_2 + k_3x_3$, last $k_1x_7 + k_2x_8 + k_3x_9$.

(b) **1D, padding 1, stride 2.** Output dim $\lfloor (9 + 2 - 3)/2 \rfloor + 1 = 5$. First $k_1 \cdot 0 + k_2x_1 + k_3x_2$, second $k_1x_2 + k_2x_3 + k_3x_4$.

(c) **Matrix form for part (a).** $K \in \mathbb{R}^{7 \times 9}$, each row i has (k_1, k_2, k_3) at columns $(i, i + 1, i + 2)$:

$$K = \begin{pmatrix} k_1 & k_2 & k_3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & k_1 & k_2 & k_3 & 0 & 0 & 0 & 0 & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & k_1 & k_2 & k_3 \end{pmatrix}.$$

Toeplitz / banded.

(d) **2D, $K = \frac{1}{4} \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$, $X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$, no pad, stride 1.** Output 2×2 : $(3, 4; 6, 7)$. This is a 2×2 box / averaging filter.

(e) **General formula.** $W' = W - K + 1$ for stride 1, no pad; with stride s and padding p , $W' = \lfloor (W + 2p - K)/s \rfloor + 1$.

Discussion 10 Q1: Q/K/V mechanics, full numerical

Setup. $\mathbf{x}_1 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$, $\mathbf{x}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$, $\mathbf{x}_3 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$; $\mathbf{W}_Q = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -3 \end{pmatrix}$, $\mathbf{W}_K = \begin{pmatrix} 0 & 0 & -1 \\ -2 & 0 & 1 \end{pmatrix}$, $\mathbf{W}_V = \begin{pmatrix} 8 & 0 \\ 0 & 9 \end{pmatrix}$. Convention: $\mathbf{q}_i = \mathbf{W}_Q^\top \mathbf{x}_i$.

(a) **Compute Q, K, V.** $\mathbf{W}_Q^\top = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -3 \end{pmatrix}$ etc.

$$\begin{array}{lll} \mathbf{q}_1 = (2, 3, -5)^\top, & \mathbf{q}_2 = (-1, 0, -2)^\top, & \mathbf{q}_3 = (1, -2, 8)^\top. \\ \mathbf{k}_1 = (-6, 0, 1)^\top, & \mathbf{k}_2 = (0, 0, 1)^\top, & \mathbf{k}_3 = (4, 0, -3)^\top. \\ \mathbf{v}_1 = (16, 27)^\top, & \mathbf{v}_2 = (-8, 0)^\top, & \mathbf{v}_3 = (8, -18)^\top. \end{array}$$

(b) **Attention from token 3.** $z_{3,1} = \mathbf{q}_3^\top \mathbf{k}_1 = 1(-6) + (-2)(0) + 8(1) = 2$. $z_{3,2} = 1(0) + (-2)(0) + 8(1) = 8$. $z_{3,3} = 1(4) + (-2)(0) + 8(-3) = -20$.

(c) **Output for token 3.** With softmax weights $(0.00247, 0.99753, 6.9 \times 10^{-13})$: $\mathbf{o}_3 = 0.00247 \cdot (16, 27) + 0.99753 \cdot (-8, 0) + 6.9 \times 10^{-13} \cdot (8, -18) \approx (-7.94, 0.067)^\top$.

(d) **Matrix form.** Stack $\mathbf{x}_i^\top$ as rows of $\mathbf{X} \in \mathbb{R}^{3 \times 2}$. Then $\mathbf{q}_i^\top = \mathbf{x}_i^\top \mathbf{W}_Q$ gives $\mathbf{Q} = \mathbf{X} \mathbf{W}_Q \in \mathbb{R}^{3 \times 3}$. Similarly $\mathbf{K} = \mathbf{X} \mathbf{W}_K \in \mathbb{R}^{3 \times 3}$, $\mathbf{V} = \mathbf{X} \mathbf{W}_V \in \mathbb{R}^{3 \times 2}$.

(e) **$(\mathbf{Q} \mathbf{K}^\top)_{ij}$.** The (i, j) entry of $\mathbf{Q} \mathbf{K}^\top$ is the inner product of row i of $\mathbf{Q}$ (which is $\mathbf{q}_i^\top$) and column j of $\mathbf{K}^\top$ (which is $\mathbf{k}_j$), i.e. $\mathbf{q}_i^\top \mathbf{k}_j = z_{ij}$.

(f) **Softmax then V.** Row i of $\mathbf{A} \mathbf{V}$ equals $\sum_j A_{ij} \mathbf{v}_j^\top$, exactly the per-token weighted sum from part (c).

Discussion 10 Q2: $\sqrt{d_k}$ Scaling Derivation

For $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ iid $\mathcal{N}(\mu, \sigma^2 I)$:

- (a) **Mean.** $\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = \sum_i \mathbb{E}[q_i k_i] = \sum_i \mathbb{E}[q_i] \mathbb{E}[k_i] = d_k \mu^2$.
- (b) **Variance for $\mu = 0, \sigma = 1$.** $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] - 0 = 1$. By independence of the $q_i k_i$ products, $\text{Var}(\mathbf{q}^\top \mathbf{k}) = \sum_i 1 = d_k$.
- (c) **Scaling factor.** $\mathbb{E}[\mathbf{q}^\top \mathbf{k}/s] = 0$ trivially. $\text{Var}(\mathbf{q}^\top \mathbf{k}/s) = d_k/s^2 = 1$ forces $s = \sqrt{d_k}$.

Discussion 11 Q1: RoPE relative-position invariance

- (a) **Why PE.** Self-attention is permutation-equivariant; without positional info “the man chased the dog” and “the dog chased the man” look the same.
- (b) **Relative vs absolute.** Absolute PE assigns each position $n \in \{1, \dots, N\}$ a unique encoding $\mathbf{r}_n$ added to $\mathbf{x}_n$. Relative PE makes attention depend only on the offset $n-i$ between query position n and key position i .
- (c) **RoPE proof.** $\text{RoPE}(\mathbf{x}, m)^\top \text{RoPE}(\mathbf{y}, n) = (\mathbf{R}(m\theta)\mathbf{x})^\top (\mathbf{R}(n\theta)\mathbf{y}) = \mathbf{x}^\top \mathbf{R}(m\theta)^\top \mathbf{R}(n\theta)\mathbf{y}$. Using $\mathbf{R}(\alpha)^\top = \mathbf{R}(-\alpha)$ and $\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha + \beta)$, this is $\mathbf{x}^\top \mathbf{R}((n-m)\theta)\mathbf{y}$ — depends only on $n-m$. For shift k : $\text{RoPE}(\mathbf{x}, m+k)^\top \text{RoPE}(\mathbf{y}, n+k) = \mathbf{x}^\top \mathbf{R}((n+k)-(m+k)\theta)\mathbf{y} = \mathbf{x}^\top \mathbf{R}((n-m)\theta)\mathbf{y}$, identical.

Discussion 11 Q2: Attention heatmap matching

Pre-softmax matrices A, B, C, D map to heatmaps 3, 1, 2, 4 respectively:

- **A (constant 2):** softmax-normalizes to uniform $1/4$. Heatmap 3 (all equal).
 - **B (10 on diag, 1 off):** softmax gives diag $\frac{e^{10}}{e^{10} + 3e^1} \approx 0.9997$, off-diag ≈ 0.0001 . Heatmap 1 (sharp diagonal).
 - **C (lower triangular, $-\infty$ above):** causal mask. Each row uniform on $\{1, \dots, i\}$. Heatmap 2.
 - **D (single off-diagonal 5, rest $-\infty$):** each row attends to one specific column. Heatmap 4 (permutation pattern).
- (b) **Why C is a causal mask.** Token i may not see future tokens $j > i$. Setting upper triangle to $-\infty$ ensures softmax sends those weights to 0. Used in autoregressive (decoder-only) LMs at training so parallel computation respects the no-peeking rule.

- (c) **Temperature T on Matrix B.** With $\text{softmax}(\mathbf{x}/T)$ on a row (10, 1, 1, 1):

$$a_{\text{diag}} = \frac{e^{10/T}}{e^{10/T} + 3e^{1/T}}, \quad a_{\text{off}} = \frac{e^{1/T}}{e^{10/T} + 3e^{1/T}}.$$

$T \rightarrow 0^+$: $e^{10/T} \gg e^{1/T}$, so $a_{\text{diag}} \rightarrow 1$, $a_{\text{off}} \rightarrow 0$ (sharp). $T \rightarrow \infty$: $e^{10/T}, e^{1/T} \rightarrow 1$, so $a_{\text{diag}} \rightarrow 1/4$, $a_{\text{off}} \rightarrow 1/4$ (uniform).

Discussion 11 Q3: KV caching counts

- (a) **$\mathbf{K}_1, \mathbf{V}_1$ recomputed how many times?** At iteration n the model needs $\mathbf{K}_{1:n}, \mathbf{V}_{1:n}$, which means $\mathbf{K}_1, \mathbf{V}_1$ are recomputed at every iteration $n = 1, \dots, N$. Total: N times.

(b) **Total key projections without caching.** At iteration n , compute $\mathbf{K}_1, \dots, \mathbf{K}_n$, which is n multiplications. Sum: $\sum_{n=1}^N n = N(N+1)/2 = \mathcal{O}(N^2)$.

(c) **Total with caching.** $\mathbf{K}_n$ computed once, at step n . Total: N .

(d) **Why important for chatbots.** A long conversation history (thousands of tokens) would otherwise be reprocessed at every new turn, making latency catastrophic. Caching saves the work — only the new tokens’ K/V are computed.

Discussion 12 Q1: VLM projection layer

(a) **Dimensions.** If we left-multiply $\mathbf{H}_{\text{vis}} \in \mathbb{R}^{1024}$ by $\mathbf{W}_P$ to land in $\mathbb{R}^{4096}$: $\mathbf{W}_P \in \mathbb{R}^{4096 \times 1024}$.

(b) **Why freeze backbones.** (i) *Catastrophic forgetting*: vision encoder + LLM are large foundation models trained on massive datasets; small image-text SFT data risks degrading their general capabilities. (ii) *Compute / memory*: gradients through billion-parameter models are hugely expensive; training only $\mathbf{W}_P$ (a few million params) is orders of magnitude cheaper.

(c) **[CLS]-only vs all patch tokens.** [CLS]: highly efficient (1 token) but loses spatial detail. All patches: preserves spatial info (essential for grounding/description) but uses $\sim 256+$ tokens of context per image. Modern VLMs use all patches.

(d) **Why SFT is necessary.** The vision and language encoders were trained independently. Their embedding spaces are not aligned semantically — the “cat” direction in vision is unrelated to the “cat” embedding in the LLM. SFT teaches $\mathbf{W}_P$ to align them.

Discussion 12 Q2: SSL pretext-task table

Method	Input	Pretext	Type	Loss
Autoencoder	image	reconstruct input	generative	MSE
Context encoder	masked image	predict missing	generative	MSE + adversarial
Image rotation	rotated image	predict $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$	discriminative	cross-entropy
SimCLR	two augmentations	same image?	discriminative	contrastive

(b)(i) **Pure L_2 artifact:** blurry inpainting. The conditional mean of multiple plausible completions averages them, and the average is smooth.

(b)(ii) **Why adversarial helps:** the discriminator forces the generator to pick *one* sharp realistic mode, breaking the averaging symmetry.

Discussion 12 Q3: AR vs. diffusion

(a) **Training efficiency.** Both models train on a *single* local objective per step, decoupled from the full inference chain:

- AR: predict w_n given $w_{<n}$. One forward pass + one CE loss; no backprop through the entire sequence.
- Diffusion: predict the noise ϵ at one random timestep t . One denoising pass + one MSE loss; no backprop through all T denoising steps.

The deep multi-step graph appears only at inference, where gradients aren’t needed.

(b) Frequency content during diffusion sampling. Coarse-to-fine: at high noise ($t \approx T$), the model produces global low-frequency structure (overall composition); at low noise ($t \approx 0$), high-frequency details (textures, edges) emerge. The noise level controls the resolution of detail being synthesized at each step.

HW3 Q1: Newton on logistic regression

Setup. $J(\mathbf{w}) = -\mathbf{y} \cdot \log \mathbf{s} - (\mathbf{1} - \mathbf{y}) \cdot \log(\mathbf{1} - \mathbf{s})$, $\mathbf{s} = \sigma(\mathbf{X}\mathbf{w})$.

(a) **Gradient.** Use $\nabla_{\mathbf{w}}(\mathbf{y} \cdot \mathbf{z}) = (\nabla_{\mathbf{w}}\mathbf{z})\mathbf{y}$ when $\mathbf{y}$ is constant. Apply chain rule for $\nabla_{\mathbf{w}} \log \mathbf{s}(\mathbf{X}\mathbf{w})$: $\nabla_{\mathbf{w}}\mathbf{X}\mathbf{w} = \mathbf{X}^\top$, $\nabla_{\mathbf{x}}\mathbf{s}(\mathbf{x}) = \text{diag}(s_i(1-s_i))$, $\nabla_{\mathbf{s}} \log \mathbf{s} = \text{diag}(1/s_i)$. Compose:

$$\nabla_{\mathbf{w}} \log \mathbf{s}(\mathbf{X}\mathbf{w}) = \mathbf{X}^\top \text{diag}(s_i(1-s_i)) \text{diag}(1/s_i) = \mathbf{X}^\top \text{diag}(1-s_i).$$

Similarly $\nabla_{\mathbf{w}} \log(\mathbf{1} - \mathbf{s}(\mathbf{X}\mathbf{w})) = -\mathbf{X}^\top \text{diag}(s_i)$. Combining:

$$\begin{aligned} \nabla_{\mathbf{w}} J &= -\mathbf{X}^\top \text{diag}(1-s_i)\mathbf{y} + \mathbf{X}^\top \text{diag}(s_i)(\mathbf{1} - \mathbf{y}) \\ &= \mathbf{X}^\top \text{diag}(s_i)\mathbf{1} - \mathbf{X}^\top [\text{diag}(1-s_i) + \text{diag}(s_i)]\mathbf{y} \\ &= \mathbf{X}^\top \mathbf{s} - \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{s} - \mathbf{y}). \end{aligned}$$

(b) **Hessian.** Differentiate $\nabla J = \mathbf{X}^\top \mathbf{s}(\mathbf{X}\mathbf{w}) - \mathbf{X}^\top \mathbf{y}$ w.r.t. $\mathbf{w}$. The constant term vanishes. For the first term, use $\nabla_{\mathbf{x}} \mathbf{C}\mathbf{y}(\mathbf{x}) = (\nabla_{\mathbf{x}}\mathbf{y}(\mathbf{x}))\mathbf{C}^\top$ with $\mathbf{C} = \mathbf{X}^\top$:

$$\nabla_{\mathbf{w}}^2 J = \mathbf{X}^\top \text{diag}(s_i(1-s_i)) \mathbf{X} = \mathbf{X}^\top \mathbf{\Omega} \mathbf{X}, \quad \mathbf{\Omega} = \text{diag}(s_i(1-s_i)) \succeq 0.$$

(c) **Newton update.** $\mathbf{w}^{t+1} = \mathbf{w}^t - (\nabla^2 J)^{-1} \nabla J = \mathbf{w}^t + (\mathbf{X}^\top \mathbf{\Omega} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \mathbf{s})$.

HW3 Q2: Coordinate Descent

Setup. $g(\mathbf{w}; i) = d \langle \nabla f(\mathbf{w}), \mathbf{e}_i \rangle \mathbf{e}_i$, $i \sim \text{Unif}\{1, \dots, d\}$.

(a) **Unbiased.** $\mathbb{E}_i[g(\mathbf{w}; i)] = \frac{1}{d} \sum_{i=1}^d d \mathbf{e}_i \mathbf{e}_i^\top \nabla f(\mathbf{w}) = I \nabla f(\mathbf{w}) = \nabla f(\mathbf{w})$.

(b) **Coordinate-wise smoothness.** By Taylor with mean-value:

$$f(\mathbf{y}) = f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{1}{2} (\mathbf{y} - \mathbf{x})^\top \nabla^2 f(\tilde{\mathbf{z}}) (\mathbf{y} - \mathbf{x}).$$

Since $\mathbf{y} - \mathbf{x}$ is supported only on coordinate i , write $\mathbf{y} - \mathbf{x} = (y_i - x_i)\mathbf{e}_i$. Then $(\mathbf{y} - \mathbf{x})^\top \nabla^2 f(\tilde{\mathbf{z}}) (\mathbf{y} - \mathbf{x}) = (y_i - x_i)^2 (\nabla^2 f(\tilde{\mathbf{z}}))_{ii} \leq L_i \|\mathbf{y} - \mathbf{x}\|^2$.

(c) **Convergence.** Plugging $\mathbf{y} = \mathbf{w}_{t+1} = \mathbf{w}_t - \alpha d \langle \nabla f(\mathbf{w}_t), \mathbf{e}_{i_t} \rangle \mathbf{e}_{i_t}$ into the bound and taking $\mathbb{E}[\mathbf{e}_{i_t} \mathbf{e}_{i_t}^\top] = \frac{1}{d} I$:

$$\mathbb{E}[f(\mathbf{w}_{t+1})] \leq \mathbb{E} \left[f(\mathbf{w}_t) + \left(-\alpha + \frac{L_{\max} \alpha^2 d}{2} \right) \|\nabla f(\mathbf{w}_t)\|^2 \right].$$

Choose $\alpha = 1/(L_{\max} d)$ to make the bracket $-1/(2L_{\max} d)$, giving $\mathbb{E}[f(\mathbf{w}_{t+1})] \leq \mathbb{E}[f(\mathbf{w}_t)] - \frac{1}{2dL_{\max}} \mathbb{E} \|\nabla f(\mathbf{w}_t)\|^2$. Combined with $\frac{1}{2} \|\nabla f\|^2 \geq m(f - f^*)$: $\mathbb{E}[f(\mathbf{w}_t) - f^*] \leq (1 - m/(dL_{\max}))^t \mathbb{E}[f(\mathbf{w}_0) - f^*]$.

(d) **Quadratic case.** For $f = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w}$, $\nabla f(\mathbf{w}) = \mathbf{A} \mathbf{w} - \mathbf{b}$, so coord- i_t update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha d (\mathbf{a}_{i_t}^\top \mathbf{w}_t - b_{i_t}) \mathbf{e}_{i_t}$. $L_i = A_{ii}$, $L_{\max} = \max_i A_{ii}$, $m = \lambda_{\min}(\mathbf{A})$, $L = \lambda_{\max}(\mathbf{A})$.

(e) **Cost per iteration.** Coord descent: $\mathcal{O}(d)$ (one row dot product). Full GD: $\mathcal{O}(d^2)$ (full matrix-vector).

(f) $L_{\max} \leq L$ & **overall complexity.** $\mathbf{e}_i^\top \mathbf{A} \mathbf{e}_i = A_{ii} \leq \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \mathbf{A} \mathbf{v} = \lambda_{\max} = L$. GD: $\mathcal{O}(d^2(L/m) \log \frac{1}{\epsilon})$. Coord descent: $\mathcal{O}(dL_{\max}/m \cdot \log \frac{1}{\epsilon}) \cdot d = \mathcal{O}(d^2(L_{\max}/m) \log \frac{1}{\epsilon})$. Since $L_{\max} \leq L$, coord descent is never worse.

HW3 Q3: Backprop, Autodiff, and Implicit Diff

Setup. $\mathbf{z} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \sigma(\mathbf{z})$, $\hat{y} = \mathbf{w}_2^\top \mathbf{h} + b_2$, $L = \frac{1}{2}(\hat{y} - y)^2$.

(a) Backward through the loss/output layers.

- $\partial L / \partial \hat{y} = \hat{y} - y$.
- $\partial L / \partial \mathbf{w}_2 = (\hat{y} - y) \mathbf{h}$; $\partial L / \partial b_2 = \hat{y} - y$.
- $\partial L / \partial \mathbf{h}_j = (\hat{y} - y) (\mathbf{w}_2)_j$, vector form $\partial L / \partial \mathbf{h} = (\hat{y} - y) \mathbf{w}_2$.

(b) Through activation and first layer.

- Since $h_j = \sigma(z_j)$ elementwise, $\partial h_j / \partial z_j = \sigma'(z_j)$ and zero off-diagonal. By chain rule, $\partial L / \partial z_j = (\partial L / \partial h_j) \sigma'(z_j)$. Vector: $\partial L / \partial \mathbf{z} = (\partial L / \partial \mathbf{h}) \odot \sigma'(\mathbf{z})$.
- Since $z_j = \sum_k (\mathbf{W}_1)_{jk} x_k + (\mathbf{b}_1)_j$, $\partial z_j / \partial (\mathbf{W}_1)_{jk} = x_k$, giving $\partial L / \partial \mathbf{W}_1 = (\partial L / \partial \mathbf{z}) \mathbf{x}^\top \in \mathbb{R}^{m \times d}$. $\partial L / \partial \mathbf{b}_1 = \partial L / \partial \mathbf{z}$ (each z_j depends linearly on $(\mathbf{b}_1)_j$).

(c) **Forward count.** Total parameters: $|\mathbf{W}_1| + |\mathbf{b}_1| + |\mathbf{w}_2| + |b_2| = md + m + m + 1 = 500,500 + 500 + 500 + 1 = 501,001$ (for $d = 1000, m = 500$). Forward-mode would need 501,001 passes; reverse mode does it in 1.

(d) **JVP vs. VJP cost.** (i) Forward mode: 1 JVP per layer per basis vector $\Rightarrow nK$ total JVPs. (ii) Reverse mode: 1 VJP per layer in 1 pass $\Rightarrow K$ total. (iii) For $n = 10^6$, reverse is $10^6 \times$ cheaper.

(e) **VJP decomposition of backward pass.** Decompose forward as $\mathbf{x} \xrightarrow{f_1} \mathbf{h} \xrightarrow{f_2} \hat{y} \xrightarrow{f_3} L$.

1. Loss layer: $v_3 = 1$, $J_{f_3} = \hat{y} - y$. VJP: $\bar{\hat{y}} = \hat{y} - y$.
2. Linear layer 2: $v_2 = \hat{y}$, $J_{f_2} = \mathbf{w}_2^\top$. VJP: $\bar{\mathbf{h}} = (\hat{y} - y) \mathbf{w}_2$.
3. Activation + linear 1: $v_1 = \bar{\mathbf{h}}$, $J_{f_1} = \text{diag}(\sigma'(\mathbf{z})) \mathbf{W}_1$. VJP: $\bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \sigma'(\mathbf{z})$, then $\bar{\mathbf{x}} = \bar{\mathbf{z}}^\top \mathbf{W}_1$. Parameters' gradients are byproducts.

(f) **Implicit diff for ridge.**

- (i) $F(\mathbf{x}, \lambda) = (\mathbf{A}^\top \mathbf{A} + \lambda I) \mathbf{x} - \mathbf{A}^\top \mathbf{b} = 0$.
- (ii) $\partial_1 F = \mathbf{A}^\top \mathbf{A} + \lambda I$; $\partial_2 F = \mathbf{x}$. $\partial \mathbf{x}^* / \partial \lambda = -(\mathbf{A}^\top \mathbf{A} + \lambda I)^{-1} \mathbf{x}^*$.
- (iii) Differentiate $\mathbf{x}^* = (\mathbf{A}^\top \mathbf{A} + \lambda I)^{-1} \mathbf{A}^\top \mathbf{b}$ using $\frac{d}{d\lambda} \mathbf{M}^{-1} = -\mathbf{M}^{-1} \frac{d\mathbf{M}}{d\lambda} \mathbf{M}^{-1}$ with $d\mathbf{M}/d\lambda = I$: $d\mathbf{x}^*/d\lambda = -\mathbf{M}^{-1} \mathbf{M}^{-1} \mathbf{A}^\top \mathbf{b} = -\mathbf{M}^{-1} \mathbf{x}^*$. ✓

(g) **Unroll vs. implicit.** (i) Unrolling memory: $\mathcal{O}(Td)$ (store every iterate). (ii) Implicit: $\mathcal{O}(d^2)$ to form $\mathbf{A}$ explicitly, or $\mathcal{O}(d)$ with iterative solver. Independent of T . (iii) For $d = 10^4, T = 10^3$: unroll memory $\sim 10^7$, implicit $\sim 10^4$ (iterative). Implicit also avoids backprop through 1000 GD steps.

(h) **Failure of implicit diff.** (i) Counterexample: $f(x, \theta) = (x - \theta)^4$. Min at $x^* = \theta$. $f''(x^*) = 12(x^* - \theta)^2 = 0$. Hessian zero $\Rightarrow A = \partial_1 F = 4 \cdot 3(x - \theta)^2 = 0$ at the optimum, so A^{-1} undefined. ($x^*(\theta) = \theta$ is differentiable; only the implicit formula breaks.) (ii) Near-singular Hessian $\Rightarrow \|A^{-1}\|$ huge, amplifying solver errors $\|\hat{\mathbf{x}} - \mathbf{x}^*\|$ in the Jacobian estimate.

HW3 Q4: Information Theory and Information Bottleneck

(a) **Gibbs' inequality.** Let $u = q(x)/p(x)$. By Jensen on concave $\log$: $D_{\text{KL}}(p||q) = -\mathbb{E}_p[\log u] \geq -\log \mathbb{E}_p[u] = -\log \sum_x q(x) = -\log 1 = 0$. Equality iff u is constant a.e., i.e. $p = q$.

(b)(i) $H(X) \leq \log m$. Apply Gibbs with $q = \text{uniform}$: $0 \leq D_{\text{KL}}(p||\text{Unif}) = -H(X) + \log m$, so $H(X) \leq \log m$.

(b)(ii) Source coding. Define $q(x) = 2^{-\ell(x)}/C$ where $C = \sum_{x'} 2^{-\ell(x')} \leq 1$ (Kraft). Apply Gibbs: $0 \leq \sum p(x) \log(p(x)/q(x)) = -H(X) + \mathbb{E}_p[\ell(X)] + \log C$. Since $\log C \leq 0$: $H(X) \leq \mathbb{E}_p[\ell(X)] + \log C \leq \mathbb{E}_p[\ell(X)]$.

(c) Hospital example. $p(y=1) = 0.4(0.05) + 0.2(0.15) + 0.3(0.80) + 0.1(0.95) = 0.385$. $H(Y) = H_b(0.385) \approx 0.962$.

Partition A $\{0,1\}|\{2,3\}$: $p(a) = 0.6$, $p(y=1|a) = (0.4 \cdot 0.05 + 0.2 \cdot 0.15)/0.6 = 0.05/0.6 \approx 0.083$. $p(b) = 0.4$, $p(y=1|b) = 0.335/0.4 = 0.8375$. $H(Y|\tilde{X}) = 0.6 \cdot H_b(0.083) + 0.4 \cdot H_b(0.838) \approx 0.6(0.414) + 0.4(0.640) \approx 0.504$. $I_A = 0.962 - 0.504 \approx 0.458$.

Partition B $\{0,2\}|\{1,3\}$: $p(a) = 0.7$, $p(y=1|a) = (0.02 + 0.24)/0.7 \approx 0.371$. $p(b) = 0.3$, $p(y=1|b) = (0.03 + 0.095)/0.3 \approx 0.417$. $H(Y|\tilde{X}) \approx 0.7(0.952) + 0.3(0.980) \approx 0.960$. $I_B \approx 0.962 - 0.960 \approx 0.002$.

A retains more. A groups by disease risk; B by parity (mixing low and high risk patients in each group, washing out the signal).

(d)(i) IB identity. $I(X;Y) - I(\tilde{X};Y) = H(Y|\tilde{X}) - H(Y|X)$. For deterministic $X \rightarrow \tilde{X}$, $p(\tilde{x})p(y|\tilde{x}) = \sum_{x \in \tilde{x}} p(x)p(y|x)$. Substituting and rearranging gives $\mathbb{E}_p[D_{\text{KL}}(p(y|X)||p(y|\tilde{X}))]$.

(d)(ii) Smaller KL? Partition A (since it has larger $I(\tilde{X};Y)$, and $I(X;Y)$ is fixed).

(d)(iii) Numerical check (Partition A). $H(Y|X) = 0.4H_b(0.05) + 0.2H_b(0.15) + 0.3H_b(0.80) + 0.1H_b(0.95) \approx 0.482$. $I(X;Y) \approx 0.962 - 0.482 = 0.480$. $I(X;Y) - I_A \approx 0.022$. RHS: per- x KL contributions, sum ≈ 0.04 . Both small, close (rounding).

(d)(iv) What plays the distortion role? $D_{\text{KL}}(p(y|x)||p(y|\tilde{x}))$ between true and coarsened posteriors. Notable: nobody had to specify it; it emerged from the choice of Y as relevance variable.

(e) Compression requires non-invertibility. Invertible $T = f(X) \Rightarrow I(X;T) = H(X)$, no compression. Need many-to-one mapping; in NNs, the activation function (e.g., ReLU sending all $a < 0$ to 0) provides this.

(f)(i) Sufficient T_1 doesn't imply sufficient T_2 . Counter-example: $X \sim \text{Unif}\{0,1,2,3\}$, $Y = 1\{X \geq 2\}$. Take $T_1 = X$ (sufficient). Take $T_2 = X \bmod 2$. Then given T_2 , Y is uniform on $\{0,1\}$, so $I(T_2;Y) = 0 \neq I(X;Y)$.

(f)(ii) Why depth? Depth doesn't add information; each layer can only lose info (DPI). Plausible benefits: (1) depth lets the network break a hard compression into easier sequential steps; (2) hierarchical abstraction (edges $\rightarrow$ shapes $\rightarrow$ objects); (3) Schwartz-Ziv & Tishby argue stochastic relaxation time scales exponentially with the amount of info to compress, so chunking helps.

(g)(i) $I(X;T') = H(T') - H(\epsilon)$. Markov $X \rightarrow T \rightarrow T'$ with T' a noisy copy. By chain rule for MI: $I(X;T') = H(T') - H(T'|X)$. But T' depends on X only through T , so $H(T'|X) = H(T'|T) = H(\epsilon)$ (binary symmetric channel).

(g)(ii) Behavior as $\epsilon \rightarrow 1/2$. $H(\epsilon) \rightarrow 1$, $H(T') \rightarrow 1$, so $I(X;T') \rightarrow 0$. At $\epsilon = 1/2$, T' is independent of T (and of X), carrying no information.

(g)(iii) Why irrelevant info is more fragile. Relevant information ("which side of the boundary?") is encoded in coarse, robust features (sign of activations); small noise rarely flips them. Irrelevant information ("exact distance from boundary") lives in fine activation values; small noise

easily blurs them. SGD’s compression phase exploits this asymmetry: noise destroys irrelevant detail much faster than relevant features.

HW4 Paper Questions: ResNets and Transformers

These are short reading-comprehension questions on two papers. Capsule answers below; refer to Sections 3 and 4 for the underlying theory.

ResNet questions (Q1–Q7).

Q1: Task type.

Image recognition / classification.

Q2: Problem solved.

The *degradation problem*: deeper plain networks have higher *training* error than shallower ones — not overfitting (training error worse), but optimization difficulty.

Q3: Naive deeper net.

Construct deeper-by-identity-layers from a shallow net; this should match shallow training error trivially. In practice plain networks fail to find such a configuration — SGD can’t easily learn the identity through stacks of nonlinear layers.

Q4: Residual learning.

$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. If optimal block is identity, learning $\mathcal{F} = 0$ is easier than coercing nonlinear stacks to identity. Also keeps gradient flowing via $\partial \mathbf{y} / \partial \mathbf{x} = \partial \mathcal{F} / \partial \mathbf{x} + I$.

Q5: Dimension mismatch options.

(i) Pad $\mathbf{x}$ with zeros for new channels. (ii) Use 1×1 convolution to project $\mathbf{x}$ to match.

Q6: Three shortcut options.

(1) Identity + zero-pad: no extra params. (2) Projection only at dim changes: few extra params (sweet spot, paper recommendation). (3) Projection on every block: most params, marginal gain. More projections $\Rightarrow$ more memory & compute, slightly better accuracy.

Q7: Empirical evidence.

ResNets train successfully at 50/101/152 layers; plain counterparts plateau or degrade. Figure 4 / Table 2 of the paper show training-error of deeper ResNets monotonically improving where plain nets get worse with depth.

Transformer questions (Q8–Q21).

Q8: Task.

Sequence-to-sequence transduction (machine translation).

Q9: BLEU and perplexity.

BLEU = modified n -gram precision $\times$ brevity penalty; counts overlapping n -grams. Limit: misses semantic equivalence, under-rewards good paraphrases. *Perplexity* = $\exp(L_{\text{avg}})$, effective vocabulary size. Limit: lower PPL on test text $\neq$ better usefulness.

Q10: RNN/CNN limitations.

RNN: sequential, no parallelism; gradient must flow through long chains $\Rightarrow$ vanishing. CNN: parallelizable, but receptive field grows only with depth.

Q11: “Attention” intuitively.

Selectively focusing on the relevant parts of an input (or sequence) when producing an output — like reading and only “looking at” words that matter for the current prediction.

Q12: Encoder vs. decoder.

Encoder builds a representation of the source sequence; decoder generates the target sequence one token at a time, conditioning on encoder output and previously generated tokens.

Q13: Decoder masking.

Causal mask prevents position i from seeing future positions $j > i$ during training, so the model can’t peek at the answer.

Q14: $\sqrt{d_k}$.

$\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$ for unit-variance components; dividing by $\sqrt{d_k}$ keeps softmax inputs $\mathcal{O}(1)$, preventing saturation.

Q15: Multi-head benefits.

(1) Different heads specialize in different relations (syntactic vs. semantic). (2) Ensemble effect at no extra cost.

Q16: Q/K/V sources.

(a) Encoder self-attention: Q, K, V all from encoder input. (b) Decoder self-attention: Q, K, V all from decoder input (masked). (c) Decoder cross-attention: Q from decoder, K & V from encoder output.

Q17: Train vs. inference.

Training uses *teacher forcing* (decoder input at t is the ground-truth token at $t - 1$) so all positions are computed in parallel under the causal mask. At inference, generation is autoregressive: decoder input at t is the model’s own previous output.

Q18: Why PE.

Self-attention is permutation-equivariant; without PE, two different word orders give identical outputs. PE encodes position info via a deterministic (sinusoidal) or learned vector added to the embedding.

Q19: Self-attention vs. recurrent/conv.

(Q4 in the paper). Per-layer cost $\mathcal{O}(N^2d)$ vs. $\mathcal{O}(Nd^2)$ (RNN); $\mathcal{O}(1)$ sequential ops vs. $\mathcal{O}(N)$; max path length $\mathcal{O}(1)$ vs. $\mathcal{O}(N)$. Wins on parallelism and long-range dependencies.

Q20: Vision transformers.

Cut image into $P \times P$ patches, flatten each, project to d_{model} , treat as tokens. Add positional embeddings, run through transformer encoder.

Q21: Variable input size.

ResNet: convolutions are size-agnostic, but final pooling/head fixes output dimension; usually accept any size. ViT: changing image size changes patch count, so positional-embedding table must be resized (interpolated).

HW5 Q1: InfoNCE for biological SSL

Setup. $\mathbf{z}_i = g_{\text{enc}}(\mathbf{x}_i)$, score $f(\mathbf{z}, \mathbf{c}) = \exp(\mathbf{z}^\top \mathbf{W} \mathbf{c})$, $\mathcal{L}_i = -\log \frac{f(\mathbf{z}_i^+, \mathbf{c}_i)}{f(\mathbf{z}_i^+, \mathbf{c}_i) + \sum_j f(\mathbf{z}_j^-, \mathbf{c}_i)}$.

(a) **N -way classification interpretation.** Among 1 positive and $N-1$ negatives, identify which is the true partner of the anchor context $\mathbf{c}_i$. Cross-entropy of softmax over the N candidate scores $s_j = \mathbf{z}_j^\top \mathbf{W} \mathbf{c}_i$, with the positive's index as “correct” class. Minimizing $\mathcal{L}_i$ pushes the positive's score above all negatives’.

(b) **Gradients.** Let $a_j = \exp(s_j)/Z$, $Z = \sum_k \exp(s_k)$. $\mathcal{L}_i = -s_+ + \log Z$, so: $\partial \mathcal{L}_i / \partial s_+ = -1 + a_+ = -(1 - a_+) \leq 0$ (push s_+ up). $\partial \mathcal{L}_i / \partial s_j^- = a_j \geq 0$ (push s_j^- down). The two forces pull positives together and push negatives away — the contrastive principle.

(c) **More negatives.** Larger N tightens the MI lower bound $I(\mathbf{z}^+; \mathbf{c}) \geq \log N - \mathcal{L}_i$, encouraging more informative representations. Tradeoff: requires larger batches, more memory, more compute. Mitigations: MoCo's queue, distributed contrasting.

(d) **Contrastive vs. reconstruction.** Pixel reconstruction forces capacity to model irrelevant high-frequency texture. Contrastive operates in low-dimensional latent, ignoring pixel detail $\Rightarrow$ better downstream linear-probe accuracy.

(e) **Transfer learning.** Frozen encoder + linear probe: encoder has learned task-agnostic features over millions of unlabeled examples; the head has few parameters so it doesn't overfit on small labeled sets; backbone can't suffer catastrophic forgetting. Particularly useful when labels are expensive (e.g. wet-lab annotations).

HW5: Diffusion Theory

(a) **Conditional mean is L_2 optimum.** $\mathbb{E} \|f(\mathbf{Y}) - \mathbf{X}\|^2 = \mathbb{E} \|(\mathbf{X} - \mathbb{E}[\mathbf{X}|\mathbf{Y}]) + (\mathbb{E}[\mathbf{X}|\mathbf{Y}] - f(\mathbf{Y}))\|^2$. Cross-term has expectation $\mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X}|\mathbf{Y}])^\top h(\mathbf{Y})] = 0$ by tower property. So $\mathbb{E} \|f - \mathbf{X}\|^2 = \mathbb{E} \|\mathbf{X} - \mathbb{E}[\mathbf{X}|\mathbf{Y}]\|^2 + \mathbb{E} \|\mathbb{E}[\mathbf{X}|\mathbf{Y}] - f\|^2$, and the second is minimized at $f^* = \mathbb{E}[\mathbf{X}|\mathbf{Y}]$.

(b) **Optimal denoiser.** For $\mathbf{x}_\sigma = \mathbf{x}_0 + \sigma \epsilon$, applying part (a): $\epsilon_\sigma^*(\mathbf{x}_\sigma) = \mathbb{E}[\epsilon|\mathbf{x}_\sigma]$. Use $\mathbf{x}_0 = \mathbf{x}_\sigma - \sigma \epsilon$: $\epsilon_\sigma^* = (\mathbf{x}_\sigma - \mathbb{E}[\mathbf{x}_0|\mathbf{x}_\sigma])/\sigma$. By Tweedie's formula (computing $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_\sigma]$ via score), $\epsilon_\sigma^* = -\sigma \nabla_{\mathbf{x}_\sigma} \log p_\sigma(\mathbf{x}_\sigma)$.

(c) **Continuity equation.** For a deterministic ODE flow $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$, mass conservation reads $\partial_t p_t + \nabla \cdot (\mathbf{v} p_t) = 0$. Define $\mathbf{v}(\mathbf{x}, t) = \mathbb{E}[\dot{\sigma}(t)\epsilon|\mathbf{x}_t = \mathbf{x}]$: this is the velocity averaged over noise realizations, which transports p_0 to p_t via the ODE.

HW5: Flow matching

Setup. $\mathbf{x}_t = \alpha(t)\mathbf{x}_1 + \beta(t)\mathbf{x}_0$ or $\mathbf{x}_t = h(t)\mathbf{x}_1 + (1 - h(t))\mathbf{x}_0$.

(a) **General velocity.** Differentiate w.r.t. t : $\dot{\mathbf{x}}_t = \alpha'(t)\mathbf{x}_1 + \beta'(t)\mathbf{x}_0$. For linear $\alpha(t) = t, \beta(t) = 1 - t$: $\dot{\mathbf{x}}_t = \mathbf{x}_1 - \mathbf{x}_0$ (constant), as expected.

(b)(i) **Single-function form.** $\mathbf{x}_t = h(t)\mathbf{x}_1 + (1 - h(t))\mathbf{x}_0$, so $\dot{\mathbf{x}}_t = h'(t)\mathbf{x}_1 - h'(t)\mathbf{x}_0 = h'(t)(\mathbf{x}_1 - \mathbf{x}_0)$.

(b)(ii) **Effect of h .** $h'(t)$ controls instantaneous speed. Faster $h'(t)$ at small t means the trajectory accelerates away from noise; slower $h'(t)$ near $t = 1$ means it decelerates as it approaches the data.

(b)(iii) **Three choices.**

- $h(t) = t$: $h' = 1$ constant. Constant speed.
- $h(t) = t^2$: $h'(t) = 2t$. Slow near $t = 0$, fast near $t = 1$.
- $h(t) = \sin(\pi t/2)$: $h'(t) = \frac{\pi}{2} \cos(\pi t/2)$. Fast near $t = 0$ ($h' = \pi/2$), slow near $t = 1$ ($h' = 0$).

A standalone glossary. Each entry has a one-line definition, the key formula, and a “remember” sanity check. Source pointer in italics at the end. Designed so you can scan this section alone in 20 minutes the night before the exam.

Foundations & activations

Universal Approximation Theorem. A one-hidden-layer feed-forward network with non-polynomial activation can approximate any continuous $f : K \rightarrow \mathbb{R}$ on a compact K to arbitrary accuracy. Construction: build the piecewise-linear interpolant on a δ -fine grid using ReLU hinges $\text{ReLU}(x - x_k)$ with slope-change coefficients $(m_{k+1} - m_k)$. *Remember: existence, not efficiency — width may be exponential in input dim. (§1; lec17, Dis 8 Q2)*

Activation gradients. $\sigma'(a) = \sigma(a)(1 - \sigma(a)) \leq 0.25$; $\tanh'(a) = 1 - \tanh^2(a)$; $\text{ReLU}'(a) = \mathbf{1}\{a > 0\}$; leaky ReLU has slope α on $a < 0$; softplus $h'(a) = \sigma(a)$. *Remember: sigmoid/tanh saturate $\Rightarrow$ vanishing gradients; ReLU dies if always negative. (§1)*

Softmax + cross-entropy gradient. For logits $\mathbf{a}$ and one-hot target $\mathbf{t}$, $L = -\sum_k t_k \log \text{softmax}(\mathbf{a})_k$ has the clean gradient $\partial L / \partial a_k = \text{softmax}(\mathbf{a})_k - t_k$. *Remember: the $s_i(1 - s_i)$ Jacobian factor cancels — this is why softmax+CE is the default classification head. (§1; §2)*

XOR is not linearly separable. Linear models on the four-point XOR dataset converge to $\mathbf{w} = \mathbf{0}$, $b = \frac{1}{2}$, predicting $\frac{1}{2}$ on every input. A 1-hidden-layer ReLU net with $\mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $\mathbf{c} = (0, -1)^\top$, $\mathbf{w} = (1, -2)^\top$, $b = 0$ solves it. *Remember: only nonlinearity buys non-linearly-separable functions — deep linear stacks collapse. (§1)*

Optimization & backprop

Backprop master equation. $\bar{v}_i = \sum_{c \in \text{children}(i)} \bar{v}_c (\partial v_c / \partial v_i)$, where $\bar{v} = \partial L / \partial v$. Forward pass caches values; backward pass topologically sums upstream gradients $\times$ local Jacobians. *Remember: each child sends an error signal; the node sums them. Cost linear in parameters. (§2)*

VJP vs. JVP. VJP $\mathbf{v}^\top J_f \in \mathbb{R}^{1 \times n}$ = reverse mode (backprop); JVP $J_f \mathbf{u} \in \mathbb{R}^m$ = forward mode. For scalar loss with n parameters, reverse needs 1 pass, forward needs n . *Remember: ML uses reverse mode because losses are scalar and n is huge. (§2; HW3 Q3d)*

Newton on logistic regression. With $\mathbf{s} = \sigma(\mathbf{X}\mathbf{w})$ and labels $\mathbf{y} \in \{0, 1\}^n$: $\nabla J = \mathbf{X}^\top (\mathbf{s} - \mathbf{y})$; $\nabla^2 J = \mathbf{X}^\top \mathbf{D} \mathbf{X}$, $\mathbf{D} = \text{diag}(s_i(1 - s_i)) \succeq 0$. Update $\mathbf{w}^{t+1} = \mathbf{w}^t - (\mathbf{X}^\top \mathbf{D} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{s} - \mathbf{y})$. *Remember: quadratic convergence near optimum; each step is $\mathcal{O}(d^3)$. Convex (PSD Hessian) so unique minimum. (§2; HW3 Q1)*

Gradient descent convergence. For μ -strongly convex, L -smooth f , GD with step $\alpha \in (0, 2/L)$ has rate $M = \max\{|1 - \alpha\mu|, |1 - \alpha L|\} < 1$. Optimal $\alpha^* = 2/(L + \mu)$, $M^* = (\kappa - 1)/(\kappa + 1)$, $\kappa = L/\mu$. $\mathcal{O}(\log 1/\epsilon)$ iterations to reach ϵ . *Remember: the condition number κ is the only thing that matters; ill-conditioned problems converge slowly. (§2; Dis 8 Q1)*

Coordinate descent. Pick $i \sim \text{Unif}\{1, \dots, d\}$, update $w_i \leftarrow w_i + \alpha \partial f / \partial w_i$. With $\alpha = 1/(dL_{\max})$, $L_{\max} = \max_i \sup_{\mathbf{w}} (\nabla^2 f)_{ii}$, expected progress per step $\geq \frac{1}{2dL_{\max}} \|\nabla f\|^2$. *Remember: $L_{\max} \leq L = \lambda_{\max}$; one step is $\mathcal{O}(d)$ vs. $\mathcal{O}(d^2)$ for full GD on quadratics. (§2; HW3 Q2)*

Implicit differentiation. For $\mathbf{x}^*(\boldsymbol{\theta}) = \arg \min_{\mathbf{x}} f(\mathbf{x}, \boldsymbol{\theta})$ with $\nabla_{\mathbf{x}} f(\mathbf{x}^*, \boldsymbol{\theta}) = 0$, $\partial \mathbf{x}^* / \partial \boldsymbol{\theta} = -[\nabla_{\mathbf{x}}^2 f]^{-1} \partial_{\boldsymbol{\theta}} \nabla_{\mathbf{x}} f$. *Remember: avoids backprop through T unrolled steps. Memory $\mathcal{O}(d)$ vs. $\mathcal{O}(dT)$. Fails when Hessian is singular (flat minima). (§2; HW3 Q3f-h)*

Initialization. Zero init breaks symmetry: every unit in a layer becomes identical and stays identical. Random small init: $w \sim \mathcal{N}(0, \epsilon^2)$. He init for ReLU: $\epsilon^2 = 2/n_{\text{in}}$. Xavier for tanh: $1/n_{\text{in}}$. Remember: keeps activation variance roughly constant across layers, so gradients flow. (§2)

Batch normalization. $\hat{a}_{km} = (a_{km} - \mu_m) / \sqrt{\sigma_m^2 + \delta}$ over a minibatch, followed by learnable affine $\gamma \hat{a} + \beta$. Inference uses running averages of μ, σ . Remember: train and inference behavior differ. Keeps pre-activations in the gradient sweet spot. (§2)

CNNs

Convolution output dim. $W' = \lfloor (W - K + 2p)/s \rfloor + 1$. Remember: “same” padding with stride 1 is $p = (K - 1)/2$. Stride 2 with kernel 3, pad 1 halves the dim. (§3; Dis 9)

Conv parameter count. $F \cdot k_h \cdot k_w \cdot C$ for F filters, $k_h \times k_w$ kernel, C input channels (plus F biases). Remember: depends on filters and channels, not on spatial dimensions of the image (that’s the whole point of weight sharing). (§3)

1D conv as a Toeplitz matrix. $\mathbf{y} = \mathbf{K}\mathbf{x}$, $\mathbf{K} \in \mathbb{R}^{(W-K+1) \times W}$, with each row a shifted copy of the kernel. Remember: convolution is linear in the kernel weights; weight sharing is the constraint that all rows of $\mathbf{K}$ share the same non-zero entries. (§3; Dis 9 Q1)

Receptive field. $r_L = r_{L-1} + (K_L - 1) \prod_{\ell < L} s_\ell$, $r_0 = 1$. Without strides: $1 + L(K - 1)$ (linear). With strides/pooling: multiplicative. Remember: stacking 3×3 filters with pooling gives whole-image receptive fields in $\mathcal{O}(\log)$ depth. (§3)

Backprop through CNN. Weight gradient sums over spatial positions because the filter is shared: $\partial \mathcal{L} / \partial K_{u,v} = \sum_{i,j} (\partial \mathcal{L} / \partial Y_{i,j}) X_{i+u-1, j+v-1}$. Max-pool gradient routes only to the argmax position; cache the argmax index forward. Remember: weight sharing $\Rightarrow$ accumulate; max-pool $\Rightarrow$ sparse routing. (§3)

ResNet residual block. $\mathbf{y} = \mathcal{F}(\mathbf{x}; \mathbf{W}) + \mathbf{x}$. Identity is the default; the network only learns the residual $\mathcal{F}$. Remember: $\partial \mathbf{y} / \partial \mathbf{x} = \partial \mathcal{F} / \partial \mathbf{x} + \mathbf{I}$; the $+\mathbf{I}$ keeps the gradient from vanishing. Solves the degradation problem (deeper-than-shallow training error). (§3; HW4 Q1-7)

ResNet shortcut options (HW4 Q5-6). (1) Identity + zero-pad missing channels — no extra params. (2) 1×1 projection only at dim changes — few extra params, sweet spot. (3) 1×1 projection on every block — most params, marginal gain. Remember: option 2 is the standard. (§3; HW4 Q5-6)

Double descent. Test error vs. model capacity peaks at the interpolation threshold (zero training error), then descends again as capacity grows past it. Remember: classical bias-variance gives one descent; over-parameterized nets show two. SGD implicitly selects low-norm solutions among many that fit. (§3)

Transformers

Self-attention output. $\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}_{\text{row}}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})\mathbf{V} \in \mathbb{R}^{N \times d_v}$. $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{N \times d_k}$, $\mathbf{V} \in \mathbb{R}^{N \times d_v}$. Remember: rows of the attention matrix sum to 1; A_{ij} is how much token i attends to j . (§4)

$\sqrt{d_k}$ scaling. For $q_i, k_i \sim \mathcal{N}(0, 1)$ iid, $\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = 0$, $\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$. Dividing by $\sqrt{d_k}$ gives unit-variance softmax inputs at initialization. Remember: statistical normalization, not temperature. Don’t confuse with T in $\text{softmax}(\mathbf{x}/T)$. (§4; Dis 10 Q2, HW4 Q14)

Multi-head attention. h parallel attentions with independent $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)}$ each of dim $d_k = d_{\text{model}}/h$, concatenated then re-projected by $\mathbf{W}_O$. Remember: total params $\approx$ a single-head of size d_{model} ; benefit is specialization across heads. (§4; HW4 Q15)

Sinusoidal positional encoding. $\text{PE}_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d})$, $\text{PE}_{(\text{pos}, 2i+1)} = \cos(\cdot)$. Added to embeddings, not concatenated. Remember: relative shifts are linear functions of PE via angle-addition; bounded; works for arbitrary lengths. (§4)

Rotary positional embedding (RoPE). $\text{RoPE}(\mathbf{x}_n, n) = \mathbf{R}(n\theta)\mathbf{x}_n$. Then $\text{RoPE}(\mathbf{x}, m)^\top \text{RoPE}(\mathbf{y}, n)$ depends only on $n - m$. Remember: relative-position encoding without explicit positional bias terms. Standard in modern LLMs. (§4; Dis 11 Q1)

Causal masking. Add $-\infty$ to upper-triangular entries of $\mathbf{QK}^\top$ before softmax, sending them to 0. Remember: prevents the model from peeking at future tokens during autoregressive training. Padding mask is separate. (§4; Dis 11 Q2, HW4 Q13)

Cross-attention K, V. In encoder-decoder cross-attention, queries come from the decoder, keys and values from encoder memory. Remember: “Q from where I am, K/V from what I’m looking at.” Reversing this is a standard bug. (§4; HW4 Q16)

KV caching. Without caching: $\mathcal{O}(N^2)$ key/value projections to generate N tokens. With caching: store K, V from prior steps and only compute the new $\mathbf{K}_n, \mathbf{V}_n$ — total $\mathcal{O}(N)$. Remember: 2–3 orders of magnitude inference speedup. (§4; Dis 11 Q3)

Vision transformer (ViT). Image $\rightarrow P \times P$ patches $\rightarrow$ flatten $\rightarrow$ project to d_{model} . Standard transformer encoder on patch tokens. Remember: changing input image size changes the patch count, so the PE table must be resized (interpolated). (§4; HW4 Q20-21)

Why attention beats RNN. Self-attention: $\mathcal{O}(N^2d)$ per layer, $\mathcal{O}(1)$ sequential ops, max path length $\mathcal{O}(1)$. RNN: $\mathcal{O}(Nd^2)$, $\mathcal{O}(N)$ sequential, path length $\mathcal{O}(N)$. Remember: attention is parallelizable and has constant path length between any two tokens. (§4; HW4 Q19)

LLMs

Causal LM training loss. $\mathcal{L} = -\sum_{t=1}^T \log p_\theta(w_t | w_{<t})$. Per-token cross-entropy, computed in parallel via causal mask (teacher forcing). Remember: training is fully parallel; only inference is sequential. (§5)

Perplexity. $\text{PPL} = \exp(\mathcal{L}_{\text{avg}})$. Effective vocabulary size: $\text{PPL}=20$ means as confused as picking uniformly among 20 tokens. Remember: lower PPL on web text $\neq$ better assistant. RLHF can increase PPL while making the model more useful. (§5; HW4 Q9)

LoRA (low-rank adaptation). $\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}$, $\mathbf{A} \in \mathbb{R}^{r \times d}$, $\mathbf{B} \in \mathbb{R}^{d \times r}$, $r \ll d$. Train $\mathbf{A}, \mathbf{B}$, freeze $\mathbf{W}_0$. Remember: $2dr$ params instead of d^2 ; multiple adapters swappable per task; avoids catastrophic forgetting. (§5)

RLHF objective. $\mathcal{J} = \mathbb{E}_{\mathbf{y} \sim \pi_\theta} [r_\phi(\mathbf{x}, \mathbf{y})] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{SFT}})$. Pipeline: SFT $\rightarrow$ reward model $\rightarrow$ PPO with KL anchor to SFT. Remember: KL term is essential — without it, the LLM exploits reward hacks and loses diversity. (§5)

In-context learning (ICL). Prepend examples; model output shifts toward the inferred task. No gradient updates — “learning” is in one forward pass via attention copying patterns from prompt. Remember: emergent only at scale. (§5)

VLM projection. $H_{\text{proj}} = W_P H_{\text{vis}}$, $W_P \in \mathbb{R}^{d_{\text{txt}} \times d_{\text{vis}}}$. Freeze backbones during alignment SFT; train only W_P . *Remember: matching embedding dims is necessary but not sufficient — spaces of independently-trained encoders aren't semantically aligned. (§5; Dis 12 Q1)*

SSL & generative models

Pretext-task taxonomy (Dis 12 Q2).

- Autoencoder: reconstruct input. Generative, MSE.
- Context encoder: predict masked region. Generative, MSE+adversarial.
- Image rotation: predict $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$. Discriminative, cross-entropy.
- SimCLR: same image (positive) or different (negative). Discriminative, contrastive.

Remember: generative = pixel-space loss; discriminative = label-space loss. (§6; Dis 12 Q2)

InfoNCE objective. $\mathcal{L}_i = -\log \frac{f(\mathbf{z}_i^+, \mathbf{c}_i)}{f(\mathbf{z}_i^+, \mathbf{c}_i) + \sum_j f(\mathbf{z}_j^-, \mathbf{c}_i)}$. N -way classification: identify the positive among N candidates. *Remember: also a lower bound on MI: $I(\mathbf{z}^+; \mathbf{c}) \geq \log N - \mathcal{L}_i$. Larger batches $\Rightarrow$ tighter bound. (§6; HW5 Q1)*

InfoNCE gradients. $\partial \mathcal{L}_i / \partial s_+ = a_+ - 1 < 0$ (push positive score up); $\partial \mathcal{L}_i / \partial s_j^- = a_j > 0$ (push negative score down). *Remember: positives pulled together, negatives pushed away — this is the contrastive principle. (§6; HW5 Q1b)*

Conditional mean is the L_2 optimum. $\arg \min_f \mathbb{E} \|f(\mathbf{Y}) - \mathbf{X}\|^2 = \mathbb{E}[\mathbf{X} | \mathbf{Y}]$. *Remember: this is why pure L_2 image reconstruction is blurry (it averages plausible completions). Adversarial loss breaks the symmetry. (§6)*

Optimal denoiser (Tweedie's formula). For $\mathbf{x}_\sigma = \mathbf{x}_0 + \sigma \epsilon$, $\epsilon_\sigma^*(\mathbf{x}_\sigma) = -\sigma \nabla_{\mathbf{x}_\sigma} \log p_\sigma(\mathbf{x}_\sigma)$. *Remember: noise prediction and score prediction are the same up to a $-\sigma$ factor. Diffusion = score-matching. (§6; HW5)*

Continuity equation. $\partial_t p_t + \nabla \cdot (\mathbf{v} p_t) = 0$, where $\mathbf{v}(\mathbf{x}, t)$ is the velocity of the deterministic ODE flow that pushes p_0 to p_t . *Remember: probability mass is conserved. The probability-flow ODE samples from a diffusion model by integrating $\mathbf{v}$ backward in time. (§6; HW5)*

Flow matching. For $\mathbf{x}_t = h(t)\mathbf{x}_1 + (1 - h(t))\mathbf{x}_0$, h monotone with $h(0) = 0, h(1) = 1$: $\dot{\mathbf{x}}_t = h'(t)(\mathbf{x}_1 - \mathbf{x}_0)$. *Remember: $h(t) = t$ is constant speed; $h(t) = t^2$ accelerates; $h(t) = \sin(\pi t/2)$ decelerates. Train a network to match $\mathbf{v}$; sample by integrating the ODE. (§6; HW5)*

Protein engineering & agents

Three conditioning strategies for design. (1) Train conditional generator $p_\theta(\mathbf{x} | y)$ from scratch. (2) Update unconditional $p_\theta(\mathbf{x})$ post-hoc using a property predictor (CbAS, DPO). (3) On-the-fly guidance during sampling (classifier guidance for diffusion). *Remember: All three respect Bayes' rule. Score-form $\nabla \log p(\mathbf{x} | y) = \nabla \log p(y | \mathbf{x}) + \nabla \log p(\mathbf{x})$ makes (3) clean. (§7)*

Distribution-shift pathology. $\arg \max_{\mathbf{x}} \hat{f}(\mathbf{x})$ over discrete sequence space exits the training distribution; the predictor's predictions there are unreliable. *Remember: ML-based design is naive optimization against a proxy. Mitigations: design over distributions (CbAS), KL penalty toward natural distribution, epistemic-uncertainty-aware methods. (§7)*

What is an agent? An LLM in a ReACT loop with an environment: state $\rightarrow$ LLM thought + action $\rightarrow$ tool execute $\rightarrow$ new state. Control flow is open-ended. *Remember: distinct from “LLM with a tool” (one tool call) or “workflow” (hand-coded branching tree).* (§8)

Environment (Terminal-Bench). Triple of (task description, executable Docker context, automated verifier). *Remember: the verifier is the critical piece — without it, no RL training and no benchmarking.* (§8)

GEPA prompt optimizer. Evaluate prompts on tasks via verifier; ask an LLM to reflect on failures in natural language; mutate/cross-over prompts; keep the Pareto front. *Remember: “gradient” is a natural-language reflection enabled by the verifier signal.* (§8)

Information theory & IB

Entropy bound. $H(X) \leq \log m$ for X over alphabet of size m , equality iff uniform. Proof: Gibbs $D_{\text{KL}}(p \parallel \text{Unif}) \geq 0$. *Remember: maximum entropy = uniform distribution.* (§9; HW3 Q4b)

Source coding bound. $\mathbb{E}_p[\ell(X)] \geq H(X)$ for any uniquely decodable code. Proof: define $q \propto 2^{-\ell}$ and apply Gibbs (uses Kraft $\sum 2^{-\ell(x)} \leq 1$). *Remember: entropy is the bit-rate floor for compressing X .* (§9; HW3 Q4b)

KL divergence. $D_{\text{KL}}(p \parallel q) = \sum p \log(p/q) \geq 0$, equality iff $p = q$ (Gibbs). Asymmetric. *Remember: forward KL (p outside) is mode-covering, reverse KL is mode-seeking.* (§9)

Mutual information. $I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X) = D_{\text{KL}}(p(x, y) \parallel p(x)p(y))$. Symmetric, ≥ 0 . *Remember: invariant under invertible maps; reduced by deterministic many-to-one maps.* (§9)

Data Processing Inequality. $X \rightarrow Y \rightarrow Z$ Markov $\Rightarrow I(X; Z) \leq I(X; Y)$. *Remember: processing can only lose information. This is why every deeper layer of a neural net is a candidate for compression.* (§9)

Information Bottleneck objective. $\mathcal{L} = I(X; \tilde{X}) - \beta I(\tilde{X}; Y)$. Compress $X \rightarrow \tilde{X}$ while preserving info about relevance variable Y . *Remember: $\beta \rightarrow 0$ collapses $\tilde{X}$; $\beta \rightarrow \infty$ gives no compression. Intermediate β keeps only the bits of X that matter for Y .* (§9)

IB identity. $I(X; Y) - I(\tilde{X}; Y) = \mathbb{E}_p \left[D_{\text{KL}}(p(y | X) \parallel p(y | \tilde{X})) \right]$ for deterministic $X \rightarrow \tilde{X}$. *Remember: maximizing $I(\tilde{X}; Y)$ is equivalent to minimizing expected KL between true and coarsened posteriors over Y . The IB framework discovers the right “distortion” (KL between posteriors) automatically.* (§9; HW3 Q4d)

Two phases of SGD (Schwartz-Ziv & Tishby). **Phase 1** (fitting): rapid increase in $I(T; Y)$, large consistent gradients — the network is learning to predict. **Phase 2** (compression): slow decrease in $I(X; T)$, noisy small gradients — the network is discarding irrelevant detail. *Remember: SGD’s stochastic noise drives compression. Coarse decision-boundary features are robust to noise; fine-grained features are fragile.* (§9; HW3 Q4g)